ARTEFACTOS DEL MÓDULO

Punto de Control 3 (PC3)

Fase de Elaboración: arquitectura, contratos y diseño del módulo

Pacíficos Online — IF6100

INFORMACIÓN DEL SUBGRUPO

SG1_M1_Portal_Publico	SG1_M1_Portal_Publico
Integrantes	• Arias Solís Royland. C06978 • Cortés Espinoza, Karilyn. C4E521 • Gatgens Malespin, Daniel. C4F411 • Pizarro Chaves, Benjamín Daniel. C4I460 • Ponce Juárez Roberth. C4I472
Representante ante el Comité	Roberth Ponce Juárez
Fecha de entrega	Jueves 22 de octubre de 2026
Versión del documento	V.1

1. Contrato formal del módulo (OpenAPI)

Descripción formal de las operaciones y eventos que el módulo ofrece, versionada y publicada en el repositorio.

1.1 Datos del contrato

Versión	1.1.0
Archivo en el repositorio	`contracts/m1-portal-publico/v1.1.0/openapi.yaml`
Versión de OpenAPI	3.0.3
Estado	`Publicado`
Consumidores	Frontend del portal, M5, M6 y Núcleo

1.2 Operaciones

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/ api/ m1/ v1/ productos/ busqueda	Busca productos publicados de comercios habilitados	q, id_ categoria, id_ comercio, id_ marca, precio_ minimo, precio_ maximo, orden, pagina, por_ pagina	200, ResultadoBusqueda	ERR-005, ERR-006
GET	/ api/ m1/ v1/ destacados	Comercios y productos destacados de la página principal	limite	200, Destacados	Ninguno (si M2 o M3 fallan, la sección viene vacía)
GET	/ api/ m1/ v1/ categorias	Categorías del menú con sus subcategorías	Ninguna	200, lista de CategoriaPortal	ERR-006
GET	/ api/ m1/ v1/ productos/ {id_ producto}	Página de un producto con variaciones y disponibilidad	id_ producto	200, FichaProducto	ERR-004, ERR-006
GET	/ api/ m1/ v1/ comercios	Listado de comercios habilitados	pagina, por_ pagina	200, ListadoComercios	ERR-006
GET	/ api/ m1/ v1/ comercios/ {id_ comercio}/ micrositio	Micrositio de un comercio con sus productos	id_ comercio, pagina, por_ pagina	200, Micrositio	ERR-004, ERR-006
POST	/ api/ m1/ v1/ clientes	Registra a la persona compradora y crea su cuenta en el núcleo	RegistroCliente	201, ClienteId	ERR-M1-001, ERR-005

Método	Ruta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/ api/ m1/ v1/ clientes/ {id_ cliente}	Datos de la persona compradora (solo la propia persona)	id_ cliente	200, Cliente	ERR-001, ERR-003, ERR-004
PUT	/ api/ m1/ v1/ clientes/ {id_ cliente}	Actualiza los datos de la persona compradora	id_ cliente, DatosCliente	200, ClienteId	ERR-001, ERR-003, ERR-004, ERR-M1-001, ERR-005
GET	/ api/ m1/ v1/ clientes/ {id_ cliente}/ direcciones	Direcciones de envío, la principal primero	id_ cliente	200, lista de DireccionCliente	ERR-001, ERR-003, ERR-004
POST	/ api/ m1/ v1/ clientes/ {id_ cliente}/ direcciones	Agrega una dirección de envío	id_ cliente, DatosDireccionCliente	201, DireccionCliente	ERR-001, ERR-003, ERR-004, ERR-005
PUT	/ api/ m1/ v1/ clientes/ {id_ cliente}/ direcciones/ {id_ direccion_ cliente}	Actualiza una dirección de envío	id_ cliente, id_ direccion_ cliente, DatosDireccionCliente	200, DireccionCliente	ERR-001, ERR-003, ERR-004, ERR-005

1.3 Eventos publicados

Evento	Versión	Cuándo se publica	Carga útil (campos y tipos)
Ninguno	V1.1.0	M1 no publica eventos. Solo consume los de M2 (ComercioAprobado, ComercioSuspendido y ComercioReactivado) y de M4 (ExistenciaAgotada y ExistenciaRepuesta).	—

1.4 Esquemas expuestos

- **ResultadoBusqueda.** pagina (entero), por_pagina (entero), total_resultados (entero), productos (lista de ProductoResumen).
- **ProductoResumen.** id_producto (entero), nombre (texto), imagen_principal (URL), precio (decimal), precio_hasta (decimal), moneda (texto), disponibilidad (Disponible o Agotado), comercio (id_comercio, nombre_comercial).
- **Destacados.** comercios (lista de id_comercio, nombre_comercial y logotipo), productos (lista de ProductoResumen), secciones_no_disponibles (lista).
- **CategoriaPortal.** id_categoria (entero), nombre (texto), icono_url (URL), orden_visualizacion (entero), subcategorias (lista de id_categoria y nombre).

- **FichaProducto.** id_producto, nombre, descripcion, marca, video_url, moneda, imagenes (lista de URL), variaciones (lista de VariacionFicha), comercio (id_comercio, nombre_comercial), relacionados (lista de ProductoResumen).
- **VariacionFicha.** id_variacion (entero), sku (texto), precio (decimal), atributos (lista de tipo_atributo y valor), disponibilidad (Disponible o Agotado), comprable (sí o no).
- **ListadoComercios.** pagina, por_pagina, total (enteros), comercios (lista de id_comercio, nombre_comercial y logotipo).
- **Micrositio.** comercio (id_comercio, nombre_comercial, descripcion, logo_url, banners, galeria, horarios, contacto, redes_sociales, politicas), productos (ResultadoBusqueda).
- **Cliente.** id_cliente (entero), nombre, apellidos, correo_electronico, telefono, direccion_envio (texto).
- **RegistroCliente.** Los mismos datos del cliente más contrasena, que solo se recibe, nunca se devuelve y no la guarda M1.
- **DireccionCliente.** id_direccion_cliente, id_cliente (enteros), alias_direccion, nombre_destinatario, telefono_destinatario, provincia, canton, distrito, otras_senas (texto), es_principal (sí o no).
- **Error.** error.codigo (texto), error.mensaje (texto), error.detalles (campo y mensaje, solo en ERR-005).

1.5 Historial de versiones del contrato

Versión	Fecha	Cambio	¿Rompe consumidores?
v1.0.0	05/ 10/ 2026	Publicación inicial con búsqueda, destacados y obtener, registrar y actualizar cliente.	No
v1.1.0	08/ 10/ 2026	Rutas nuevas de navegación (categorías, ficha, comercios y micrositio) y de direcciones. Métodos nuevos de la interfaz para M5 y el núcleo. Filtros nuevos de búsqueda. Registro con el núcleo y contraseña. Sesión de Laravel. Formato de error del núcleo (ERR-M1-001).	No. Los cambios de error, seguridad y registro solo afectan al frontend de M1. M5 y M6 usan la interfaz PHP, que no cambió.

2. Realización de casos de uso (mínimo dos)

Muestra cómo las clases del módulo cumplen un caso de uso, separando clases límite, control y entidad.

Se realizan los casos de uso CU-002 Buscar productos y CU-003 Consultar ficha de producto, especificados en la sección 5.2 del PC2. Se eligieron porque son los pasos 1 y 2 del recorrido de compra y porque son los que más dependen de otros módulos, ya que consultan a M2, M3 y M4.

Las interfaces hacia M2, M3 y M4 se modelan como clases límite. Así queda explícito que el portal nunca lee las tablas de otro módulo y que siempre pide la información por su contrato, como exige la regla de prohibición de lectura directa. Cada diagrama de secuencia incluye los flujos alternos del caso de uso como bloques alt y opt.

2.1 Realización del caso de uso CU-002: Buscar productos

Caso de uso (PC2)	CU-002 Buscar productos (caso de uso 1 de la sección 5.2 del PC2; requerimientos RF001 y RF002)
Actor principal	Cliente (persona compradora)
Módulos externos involucrados	M3 Catálogo, operaciones buscarProductos y listarCategorias (contrato 1.2.1). M2 Micrositios y comercios, operación ObtenerResumenComerciosLote (contrato 1.2.1). M4 Inventario, operación consultarDisponibilidadProductoLote (contrato 1.1.0).

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	PantallaBusqueda	Recibe el término y el filtro de categoría, muestra los resultados y los mensajes de los flujos A1, A2 y A3.
Control	ControlBusquedaProductos	Coordina el caso. Valida el término (RN-M1-08), pide los productos a M3, descarta los de comercios no habilitados (RN-M1-01 y RN-M1-02) y agrega la disponibilidad que entrega M4.
Límite	InterfazCatalogo	Pide a M3 los productos que coinciden con el criterio y las categorías del menú, por su contrato.
Límite	InterfazComercios	Pide a M2 el nombre y el estado de los comercios del resultado, por su contrato.
Límite	InterfazInventario	Pide a M4 si cada producto está Disponible o Agotado, por su contrato.
Entidad	CriterioBusqueda	Término, categoría y página que indicó la persona compradora. M1 lo guarda en sus tablas busquedas_usuario y filtros_aplicados.
Entidad	ResultadoBusqueda	Página de resultados con el total de coincidencias. Es una entidad de lectura, M1 la arma en cada consulta y no la guarda.
Entidad	ProductoResumen	Datos de un producto en la lista (nombre, imagen, precio, comercio y disponibilidad), tal como los entregan M3 y M4 (RN-M1-03). Es de lectura y no se guarda en M1.

Estereotipo	Clase	Responsabilidad en este caso de uso
Entidad	Categoria	Categoría o subcategoría usada para filtrar. El nombre viene de M3 y M1 guarda el orden del menú en categorias_portal.

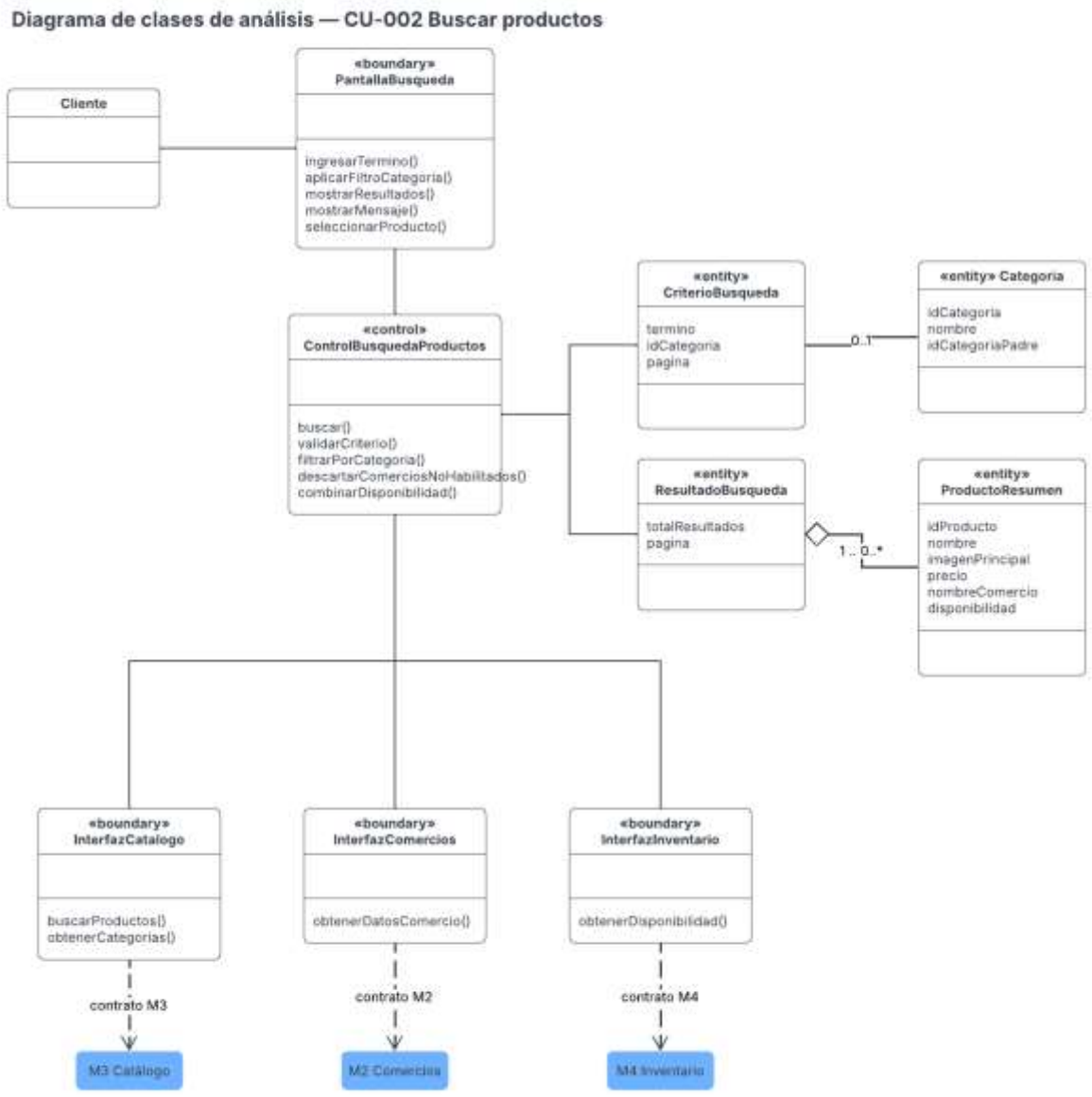


Figura 3.1. Clases de análisis de CU-002 Buscar productos.

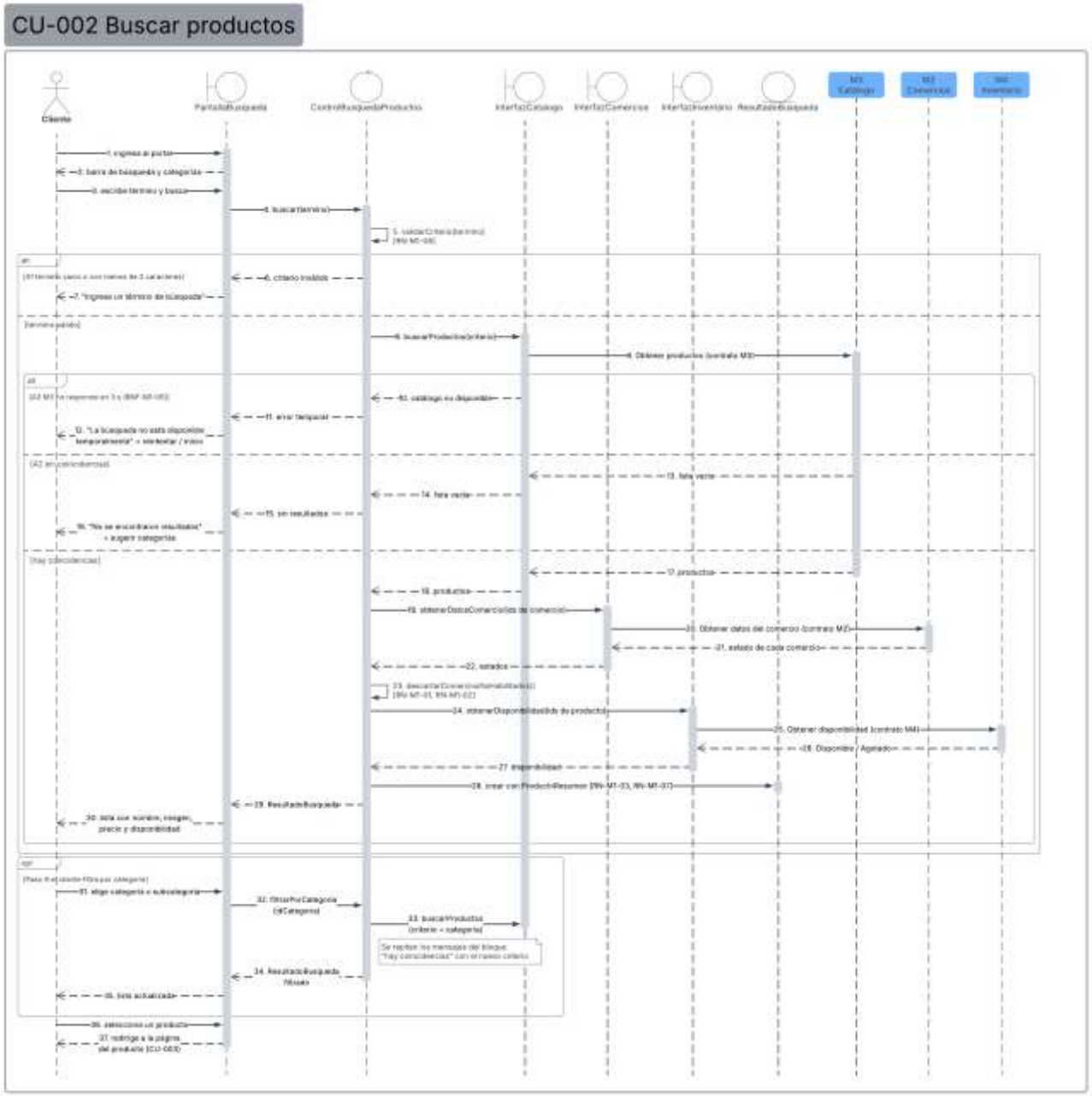


Figura 3.2. Diagrama de secuencia de CU-002 Buscar productos.

2.2 Realización del caso de uso CU-003: Consultar ficha de producto

Caso de uso (PC2)	CU-003 Consultar ficha de producto (caso de uso 2 de la sección 5.2 del PC2; requerimientos RF006 y RF008)
Actor principal	Cliente (persona compradora)
Módulos externos involucrados	M3 Catálogo, operación obtenerProducto (contrato 1.2.0). M2 Micrositios y comercios, operación ObtenerResumenComerciosLote (contrato 1.1.0). M4 Inventario, operación consultarDisponibilidadLote (contrato 1.1.0).

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	PantallaFichaProducto	Muestra la ficha, permite elegir una variación, muestra el indicador de Agotado y habilita o deshabilita la compra.
Control	ControlFichaProducto	Coordina el caso. Verifica que el comercio esté habilitado (RN-M1-02), arma la ficha sin cambiar precio ni datos (RN-M1-03) y decide si se puede comprar (RN-M1-04).
Límite	InterfazCatalogo	Pide a M3 el detalle del producto con todas sus variaciones, por su contrato.
Límite	InterfazComercios	Pide a M2 los datos y el estado del comercio vendedor, por su contrato.
Límite	InterfazInventario	Pide a M4 la disponibilidad de cada variación, por su contrato.
Entidad	FichaProducto	Datos generales del producto (nombre, descripción, imágenes, precio, marca y categoría). Es de lectura, viene de M3 y no se guarda en M1.
Entidad	VariacionProducto	Talla, color u otra presentación, con su SKU, precio, imágenes y disponibilidad. Es de lectura, viene de M3 y M4.
Entidad	ComercioVendedor	Comercio que vende el producto, con su nombre, logotipo y estado. Es de lectura y viene de M2.

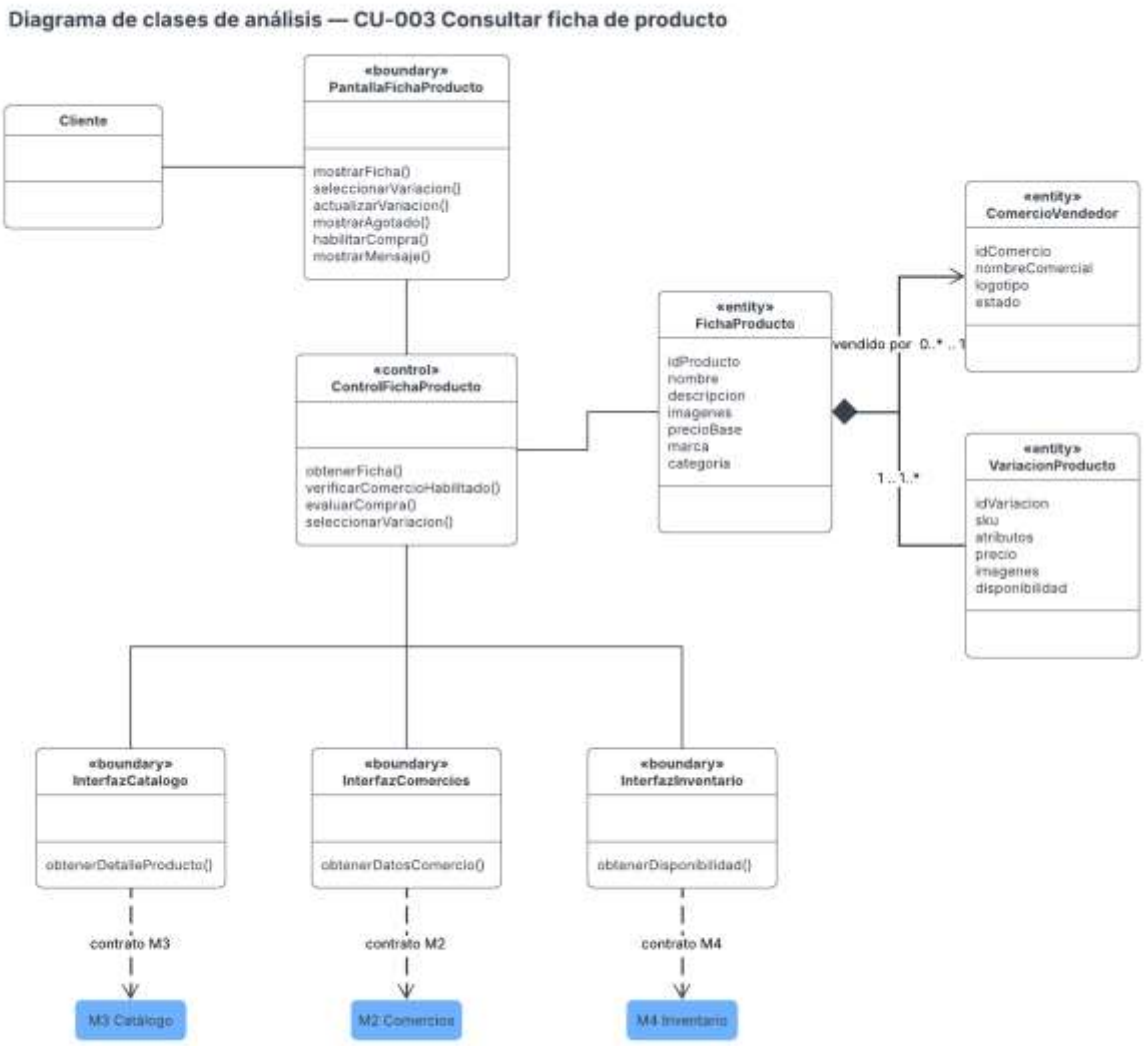


Figura 3.3. Clases de análisis de CU-003 Consultar ficha de producto.

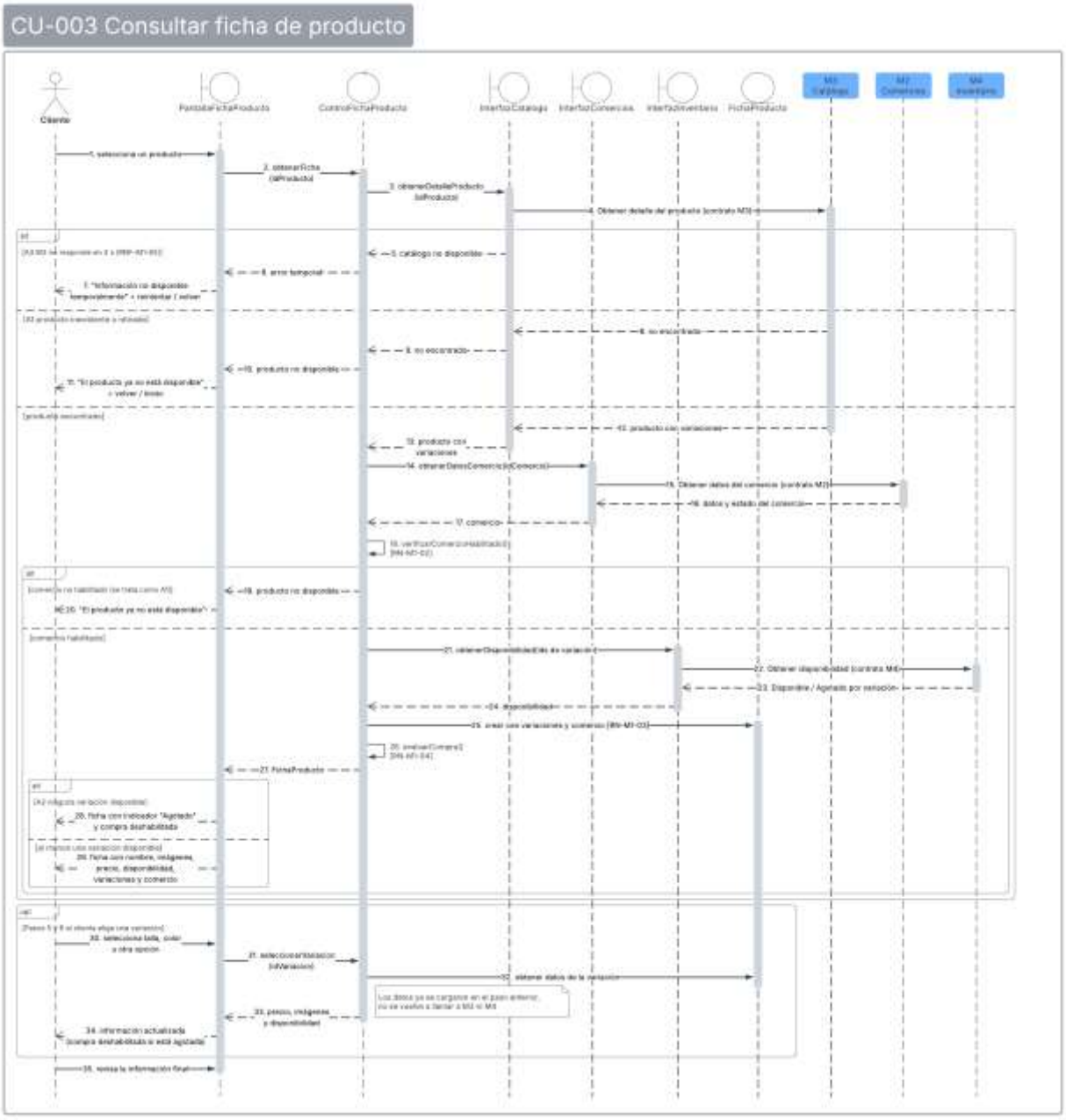


Figura 3.4. Diagrama de secuencia de CU-003 Consultar ficha de producto.

3. Diagrama de clases de diseño del módulo

Diagrama completo del módulo listo para programar: con tipos, visibilidad, firmas de métodos y navegabilidad.

El diagrama cubre todo el módulo: los casos de uso CU-001 a CU-010, las direcciones de envío del cliente, el botón Agregar al carrito y la publicación de promociones, noticias y destacados por el Encargado de la plataforma (RN-M1-10, RN-M1-11). Refina las clases de análisis de la sección 3, implementa la interfaz PHP del contrato de M1 v1.1.0 (sección 2) y sigue las capas de la vista lógica (1.5):

- **Presentación** (app/Http/Controllers/M1, app/Policies/M1 y resources/js/Pages/M1): controladores delgados que solo reciben la petición y llaman al contrato del módulo. Como el frontend es Inertia con React, los métodos que abren una pantalla devuelven Inertia\Response (Inertia::render de la página React de resources/js/Pages/M1 que corresponde a cada pantalla del prototipo de la sección 8); los que atienden las rutas /api/m1/v1 del contrato devuelven JsonResponse, y los formularios que redirigen devuelven RedirectResponse. ClientePolicy aplica RN-M1-16.
- **Aplicación** (app/Contracts/M1 y app/Services/M1): la interfaz PortalPublicoContract publicada en el contrato v1.1.0, su implementación PortalPublicoService (fachada que delega en un servicio por grupo de casos de uso), los servicios que aplican las reglas de negocio, los DTO (app/Services/M1/DTO) y las excepciones del contrato (app/Contracts/M1/Exceptions).
- **Dominio** (app/Models/M1): modelos Eloquent de las tablas de M1: clientes, direcciones_cliente, categorias_portal, productos_destacados, comercios_destacados, busquedas_usuario, filtros_aplicados, promociones_portal y noticias_portal. Los atributos de cada entidad son las columnas del modelo de datos de la sección 6 (sin created_at ni updated_at, que maneja Laravel).
- **Infraestructura** (app/Services/M1/Integraciones y app/Listeners/M1): adaptadores que llaman a los contratos de M2, M3, M4 y M5, traducen sus respuestas a DTO de M1 y cortan la espera a los 3 s (RNF-M1-04); la caché de 5 s (RNF-M1-06) y los listeners de los eventos que M1 consume de M2 y M4.

  Para que se pueda leer, el diagrama se presenta en ocho vistas. Cada clase aparece completa (con atributos y métodos) en una sola vista; en las demás aparece como un recuadro con borde punteado que solo indica la relación. Las clases en azul pertenecen a otros módulos (interfaces y eventos de M2, M3, M4 y M5) o al núcleo compartido.
- **Asociaciones:** todas llevan multiplicidad en ambos extremos. La flecha abierta indica la navegabilidad (qué clase conoce a cuál); las relaciones de agregación (rombo vacío) y composición (rombo relleno) se dibujan sin flecha.
- **Otras relaciones:** línea punteada con triángulo = realización de una interfaz; línea continua con triángulo = herencia; línea punteada con flecha = dependencia, con «use» o «listen» (escucha de un evento). Estas tres no llevan multiplicidad.
- **Clases sin asociación:** CriterioBusquedaDTO, CategoriaDTO, ClienteDTO y DireccionClienteDTO, y la clase Usuario del núcleo, no tienen líneas porque ninguna clase las guarda como atributo; solo viajan como parámetro o valor de retorno (por ejemplo, buscar(criterio: CriterioBusquedaDTO) o ver(usuario: Usuario, cliente: Cliente)). En UML esa relación es una dependencia de uso y se lee en las firmas y en la tabla 4.1.
- **Estereotipos:** «controller», «policy», «interface», «service», «adapter», «cache», «listener», «entity» (modelo Eloquent), «DTO», «enumeration», «exception», «event» (evento de otro módulo), «núcleo» (clase del núcleo compartido) y «PHP» (clase estándar del lenguaje).

Trazabilidad entre el análisis (sección 3) y el diseño

Clase de análisis	Clase de diseño	Refinamiento
PantallaBusqueda, PantallaFichaProducto (límite)	Páginas React de resources/js/Pages/M1 + BusquedaController, FichaProductoController	La pantalla vive en el frontend; el controlador la abre con Inertia y atiende las rutas del contrato llamando a PortalPublicoContract.
ControlBusquedaProductos (control)	BusquedaService	Valida el criterio, pone el nombre del comercio y descarta como salvaguarda los que no estén Activos, combina la disponibilidad y registra la búsqueda. El filtro por categoría, marca, precio u orden va en CriterioBusquedaDTO.
ControlFichaProducto (control)	FichaProductoService	Conserva obtenerFicha, verificarComercioHabilitado y evaluarCompra; la compra exige la venta habilitada en M3 y unidades en M4. La selección de una variación se resuelve en la página React.
InterfazCatalogo, InterfazComercios, InterfazInventario (límite)	CatalogoClient, ComercioClient, InventarioClient	Adaptadores sobre CatalogoServiceInterface (M3), ComerciosServiceInterface (M2) e InventarioModuleInterface (M4).
CriterioBusqueda (entidad)	CriterioBusquedaDTO + BusquedaUsuario y FiltroAplicado	El criterio viaja como DTO; lo que se guarda va a las tablas busquedas_usuario y filtros_aplicados.
Categoria (entidad)	CategoriaDTO + CategoriaPortal	El nombre viene de M3; el orden y la visibilidad del menú se guardan en categorias_portal.
ResultadoBusqueda, ProductoResumen (entidad)	ResultadoBusquedaDTO, ProductoResumenDTO	Objetos de transferencia: no se persisten en M1.
FichaProducto, VariacionProducto, ComercioVendedor (entidad)	FichaProductoDTO, VariacionDTO, ComercioDTO	Objetos de transferencia; la disponibilidad y el estado pasan a las enumeraciones Disponibilidad y EstadoComercio.
Mensajes de error de A1 y A3	CriterioInvalidoException, RecursoNoEncontradoException, DependenciaNoDisponibleException	Excepciones del contrato; el manejador del núcleo las responde con el formato de error estándar.

Diagrama de clases de diseño — Vista 1: contrato y fachada del módulo

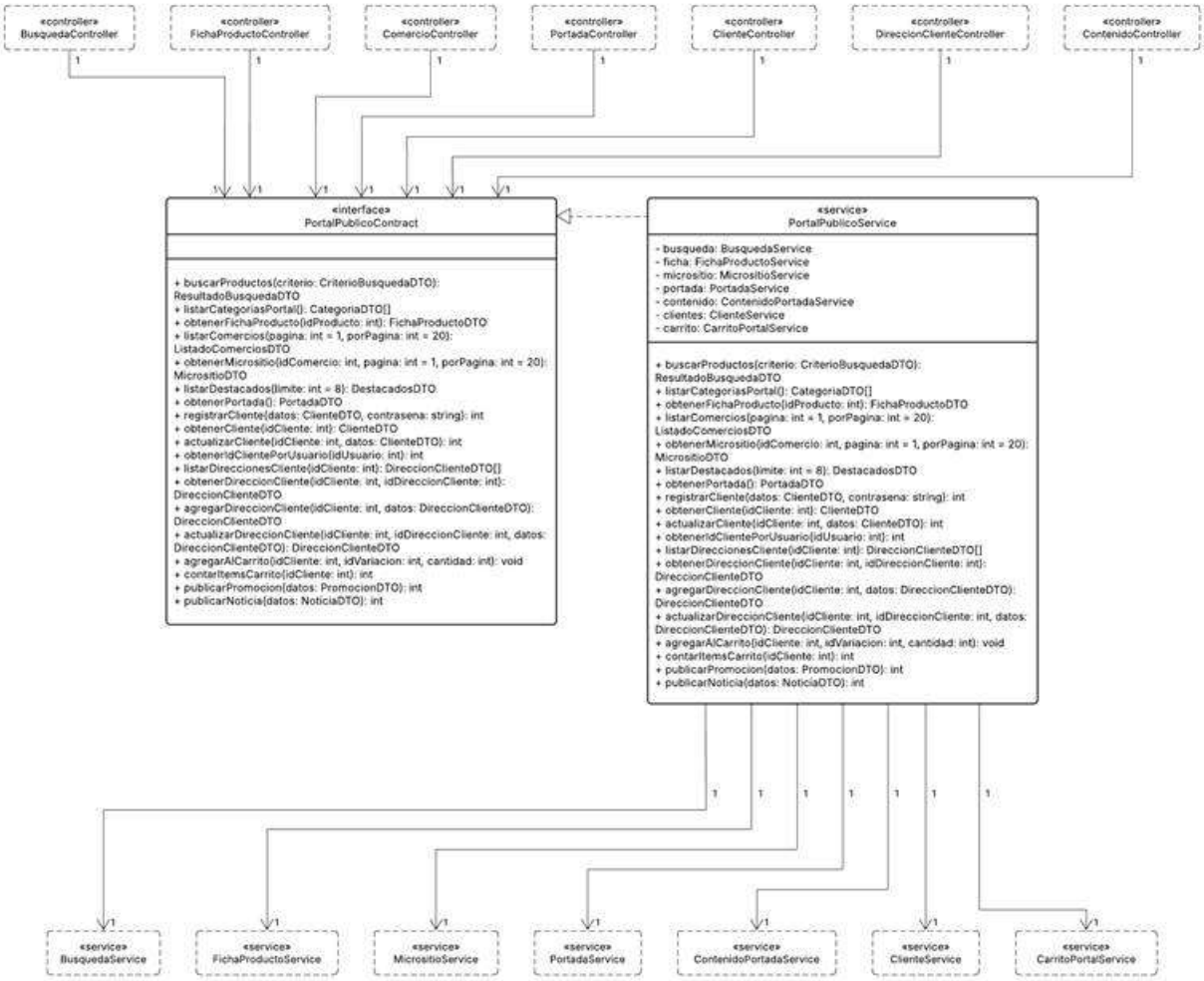


Figura 4.1. Vista 1: contrato y fachada del módulo.

Diagrama de clases de diseño — Vista 2: búsqueda, ficha de producto y carrito

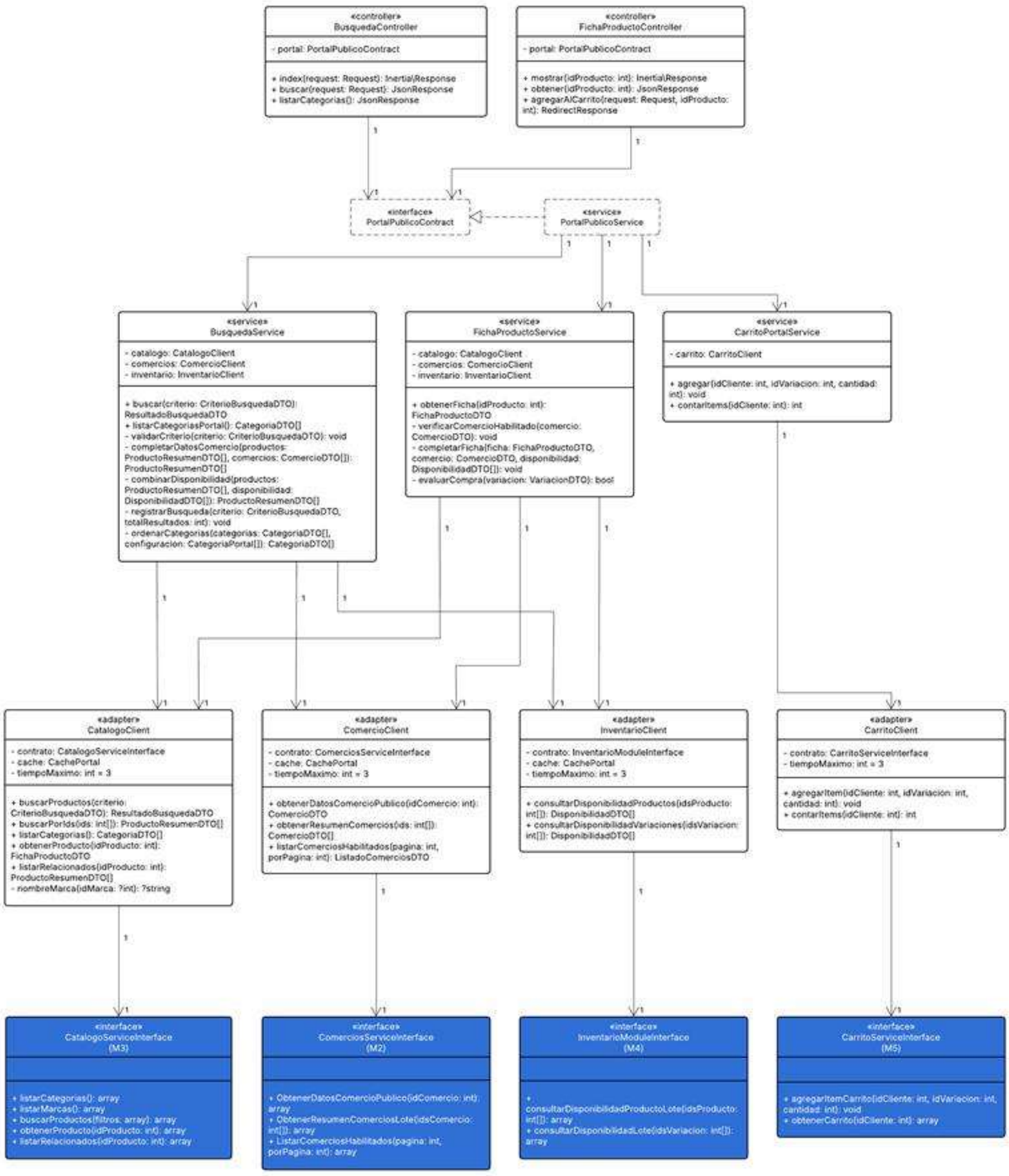


Figura 4.2. Vista 2: búsqueda, ficha de producto y carrito (CU-002, CU-003 y CU-004).

Diagrama de clases de diseño — Vista 3: portada, comercios y micrositio

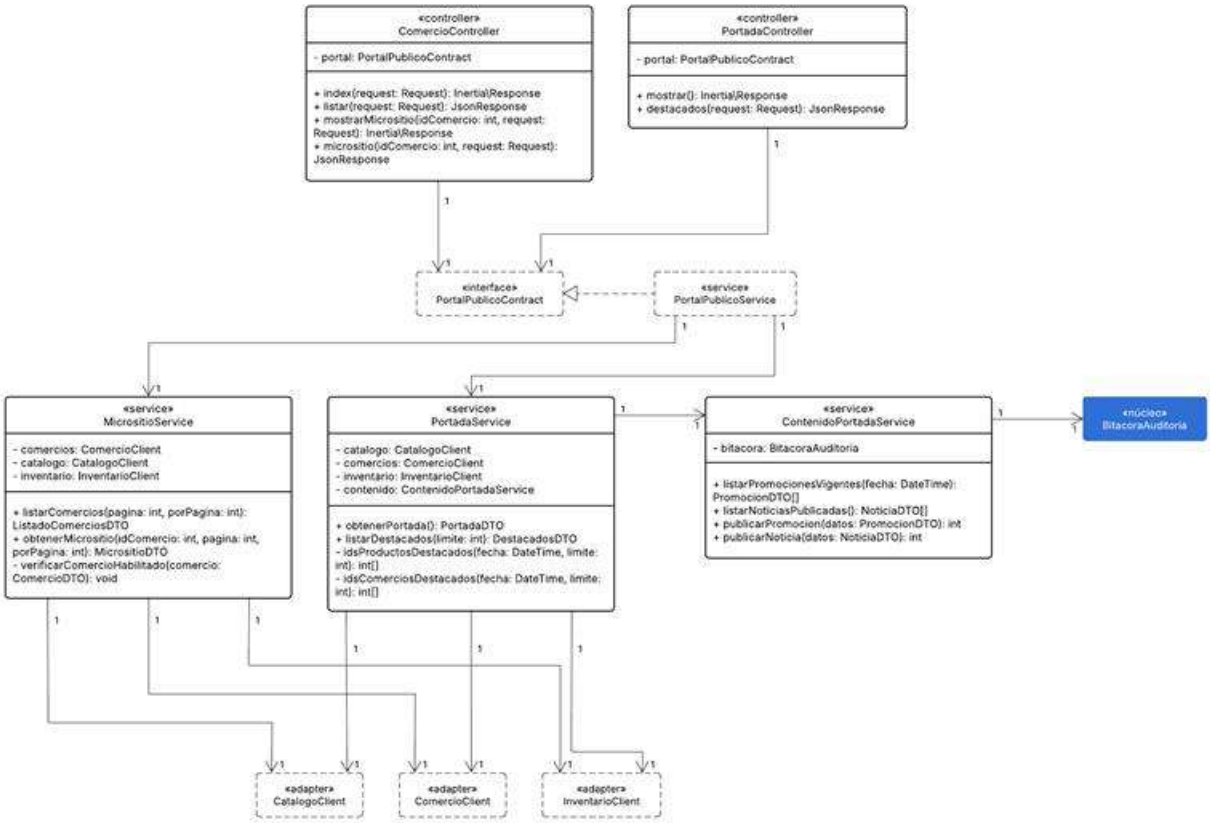


Figura 4.3. Vista 3: portada, comercios y micrositio (CU-001 y CU-005 a CU-009).

Diagrama de clases de diseño — Vista 4: cliente, direcciones y publicación de contenido

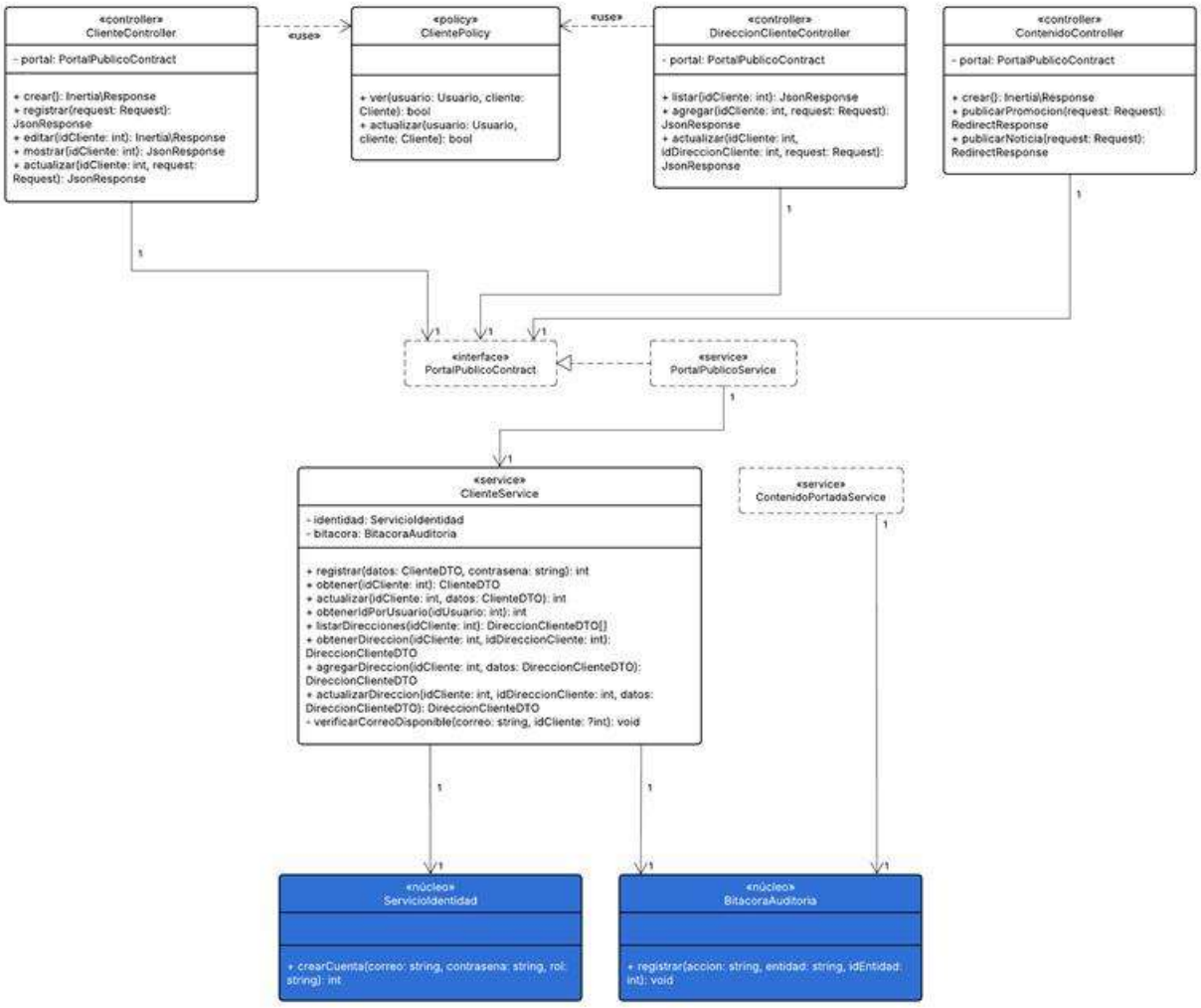


Figura 4.4. Vista 4: cliente, direcciones y publicación de contenido (CU-010, RN-M1-10, RN-M1-11 y RN-M1-16).

Diagrama de clases de diseño — Vista 5: entidades del módulo (modelos Eloquent)

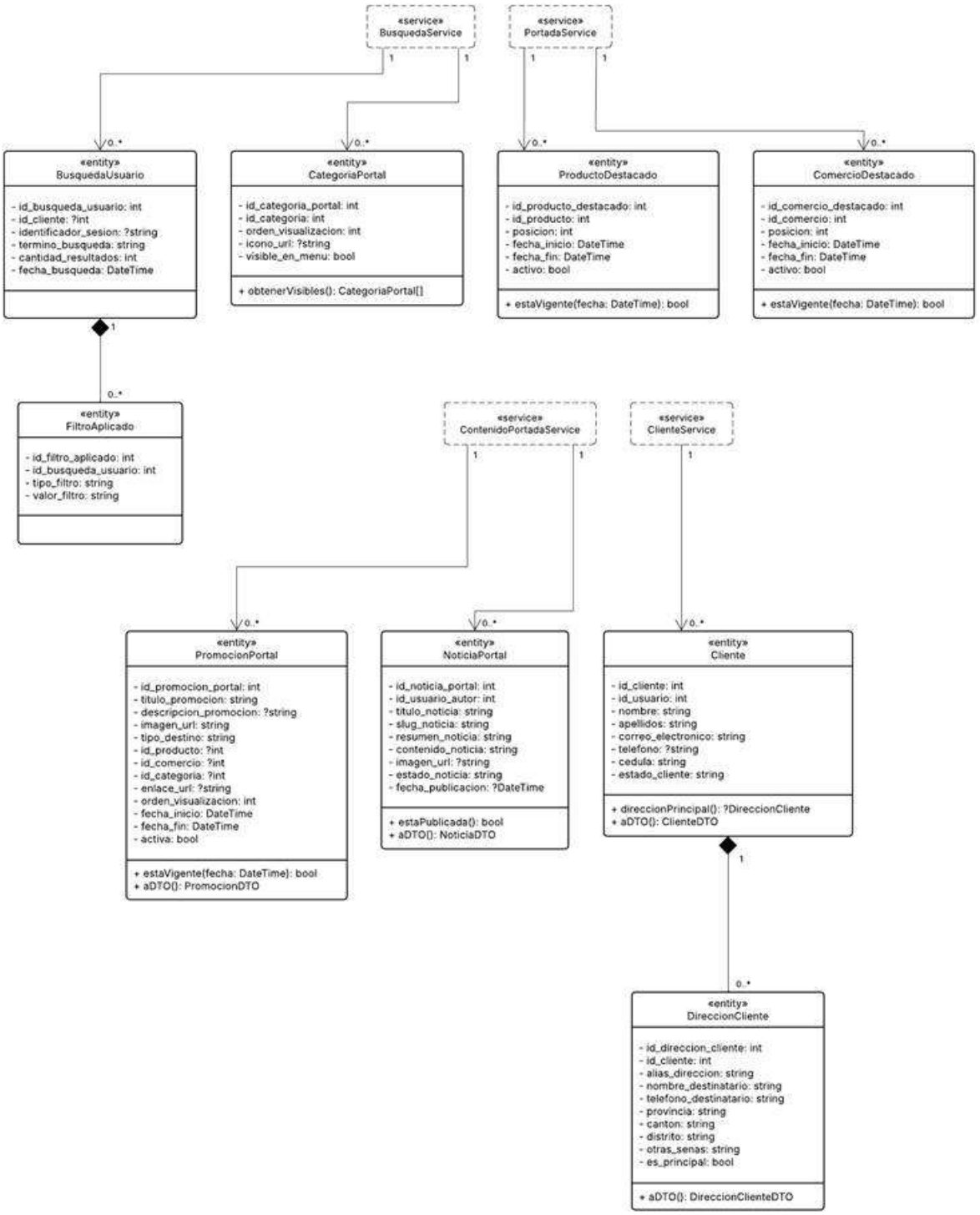


Figura 4.5. Vista 5: entidades del módulo (modelos Eloquent).

Diagrama de clases de diseño — Vista 6: objetos de transferencia (DTO) y enumeraciones

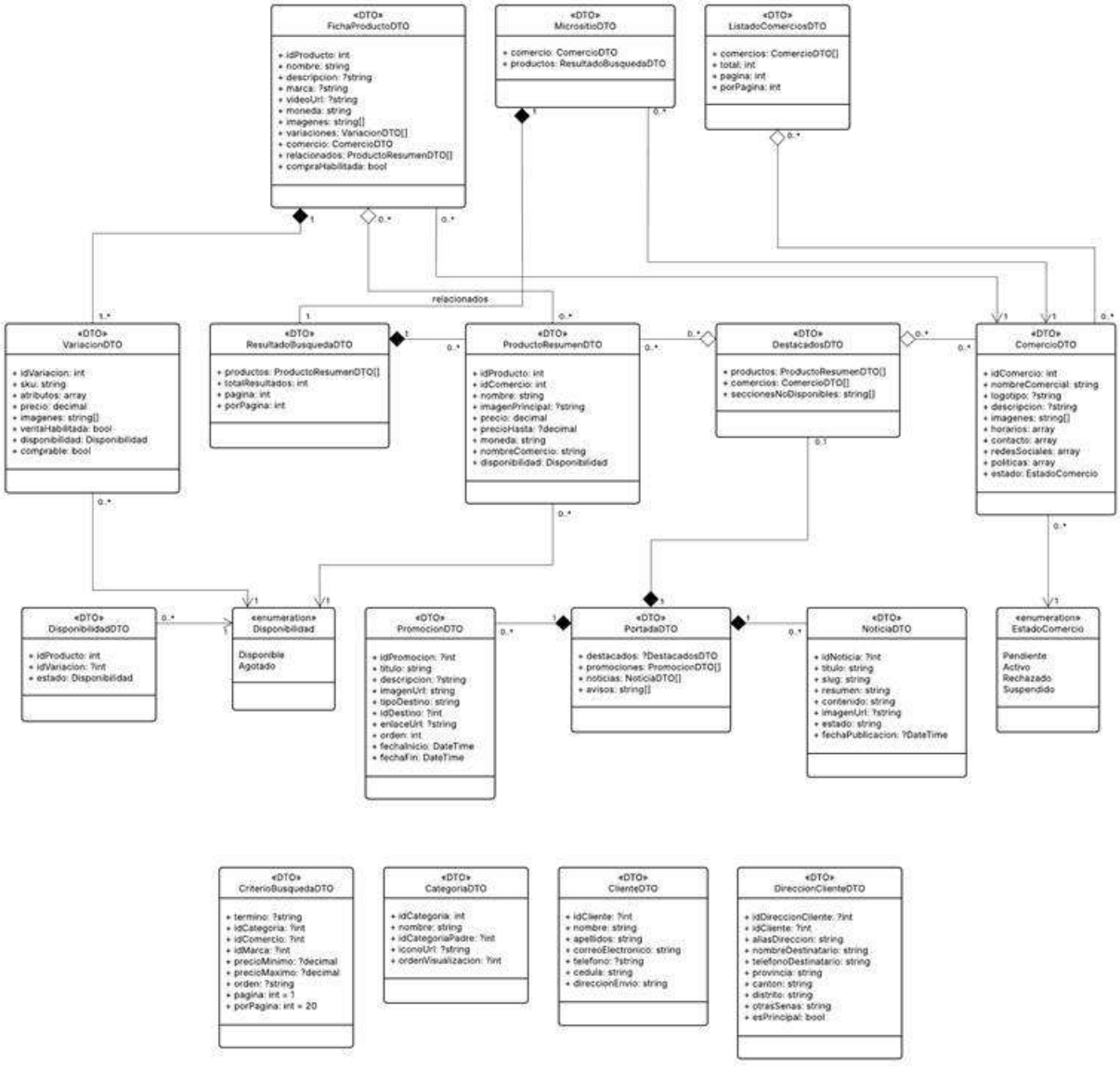


Figura 4.6. Vista 6: objetos de transferencia (DTO) y enumeraciones.

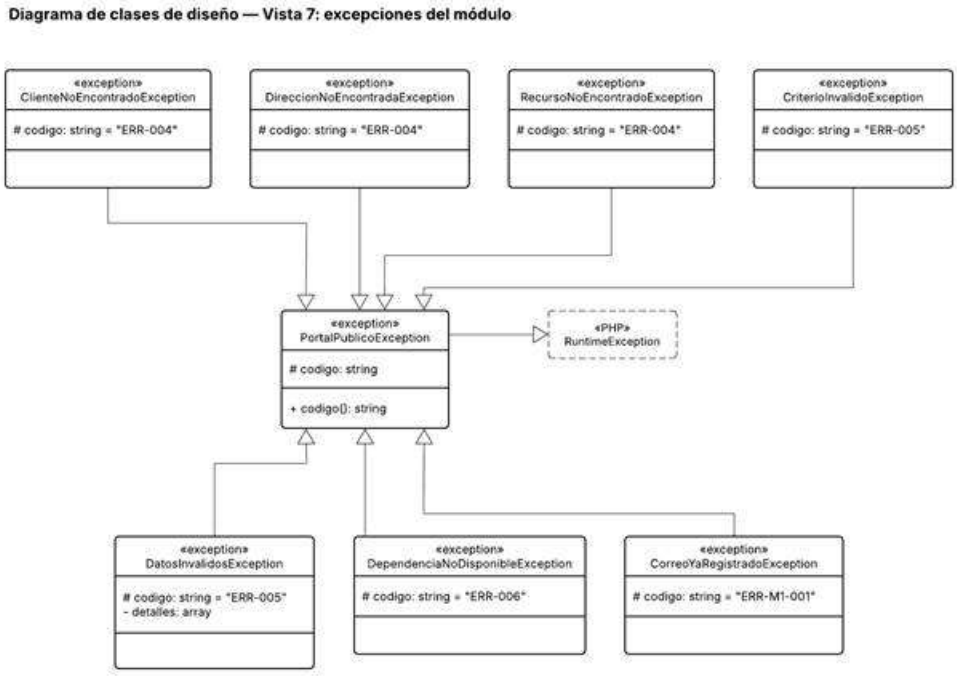


Figura 4.7. Vista 7: excepciones del módulo.

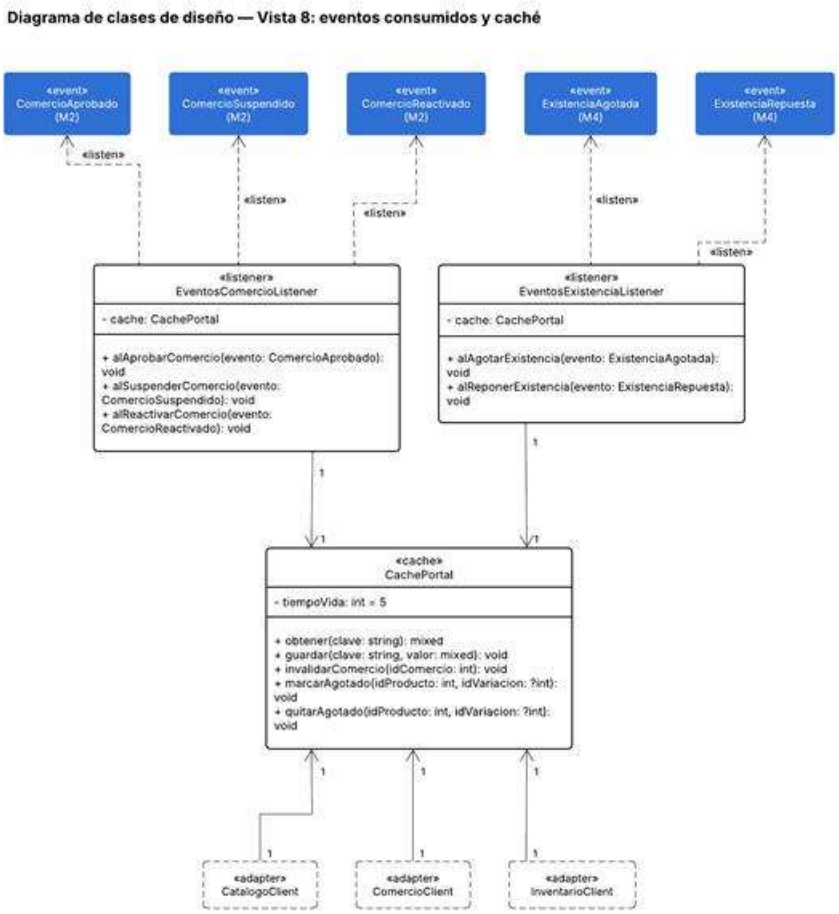


Figura 4.8. Vista 8: eventos consumidos y caché (RNF-M1-06).

3.1 Resumen de clases

Las clases se agrupan por capa. Los DTO solo tienen atributos públicos y ningún método. Las relaciones se escriben con la multiplicidad de cada extremo (origen a destino).

Clase	Capa	Atributos principales	Métodos principales (firma)	Relaciones y navegabilidad
«controller» BusquedaController Página de resultados (Inertia) y rutas GET /productos/busqueda y GET /categorias del contrato (CU-002, CU-004).	Presentación	- portal: PortalPublicoContract	+ index(request: Request): Inertia\Response + buscar(request: Request): JsonResponse + listarCategorias(): JsonResponse	Conoce a PortalPublicoContract (1 a 1).
«controller» FichaProductoController Página de la ficha (Inertia), ruta GET /productos/{id_producto} y botón Agregar al carrito (CU-003).	Presentación	- portal: PortalPublicoContract	+ mostrar(idProducto: int): Inertia\Response + obtener(idProducto: int): JsonResponse + agregarAlCarrito(request: Request, idProducto: int): RedirectResponse	Conoce a PortalPublicoContract (1 a 1).
«controller» ComercioController Páginas de comercios y de micrositio (Inertia) y rutas GET /comercios y GET /comercios/{id}/micrositio (CU-001, CU-009).	Presentación	- portal: PortalPublicoContract	+ index(request: Request): Inertia\Response + listar(request: Request): JsonResponse + mostrarMicrositio(idComercio: int, request: Request): Inertia\Response + micrositio(idComercio: int, request: Request): JsonResponse	Conoce a PortalPublicoContract (1 a 1).
«controller» PortadaController Página principal (Inertia) y ruta GET /destacados (CU-005 a CU-008).	Presentación	- portal: PortalPublicoContract	+ mostrar(): Inertia\Response + destacados(request: Request): JsonResponse	Conoce a PortalPublicoContract (1 a 1).
«controller» ClienteController Registro y cuenta del cliente (CU-010): páginas de registro y de cuenta (Inertia) y rutas POST /clientes, GET y PUT /clientes/{id}. Autoriza con ClientePolicy.	Presentación	- portal: PortalPublicoContract	+ crear(): Inertia\Response + registrar(request: Request): JsonResponse + editar(idCliente: int): Inertia\Response + mostrar(idCliente: int): JsonResponse + actualizar(idCliente: int, request: Request): JsonResponse	Conoce a PortalPublicoContract (1 a 1). Usa ClientePolicy («use»).

«controller» DireccionClienteController Rutas de direcciones de envío del contrato (GET, POST y PUT /clientes/{id}/direcciones). Autoriza con ClientePolicy.	Presentación	- portal: PortalPublicoContract	+ listar(idCliente: int): JsonResponse + agregar(idCliente: int, request: Request): JsonResponse + actualizar(idCliente: int, idDireccionCliente: int, request: Request): JsonResponse	Conoce a PortalPublicoContract (1 a 1). Usa ClientePolicy («use»).
«controller» ContenidoController Publicación de promociones y noticias por el Encargado de la plataforma (RN-M1-10, RN-M1-11).	Presentación	- portal: PortalPublicoContract	+ crear(): Inertia\Response + publicarPromocion(request: Request): RedirectResponse + publicarNoticia(request: Request): RedirectResponse	Conoce a PortalPublicoContract (1 a 1).
«policy» ClientePolicy Un cliente solo ve y edita sus propios datos y direcciones: compara cliente.id_usuario con el usuario de la sesión (RN-M1-16).	Presentación	(sin atributos)	+ ver(usuario: Usuario, cliente: Cliente): bool + actualizar(usuario: Usuario, cliente: Cliente): bool	La usa ClienteController, DireccionClienteController.
«interface» PortalPublicoContract Interfaz PHP publicada del módulo (contrato M1 v1.1.0). La usan los controladores de M1; M5 consume obtenerCliente, listarDireccionesCliente y obtenerDireccionCliente; M6, obtenerCliente; el núcleo, obtenerIdClientePorUsuario.	Aplicación	(interfaz, sin atributos)	+ buscarProductos(criterio: CriterioBusquedaDTO): ResultadoBusquedaDTO + listarCategoriasPortal(): CategoriaDTO[] + obtenerFichaProducto(idProducto: int): FichaProductoDTO + listarComercios(pagina: int = 1, porPagina: int = 20): ListadoComerciosDTO + obtenerMicrositio(idComercio: int, pagina: int = 1, porPagina: int = 20): MicrositioDTO + listarDestacados(limite: int = 8): DestacadosDTO + obtenerPortada(): PortadaDTO + registrarCliente(datos: ClienteDTO, contrasena: string): int + obtenerCliente(idCliente: int): ClienteDTO + actualizarCliente(idCliente: int, datos: ClienteDTO): int	La usa BusquedaController, ClienteController, ComercioController, ContenidoController, DireccionClienteController, FichaProductoController, PortadaController.

			+ obtenerIdClientePorUsuario(idUsuario: int): int + listarDireccionesCliente(idCliente: int): DireccionClienteDTO[] + obtenerDireccionCliente(idCliente: int, idDireccionCliente: int): DireccionClienteDTO + agregarDireccionCliente(idCliente: int, datos: DireccionClienteDTO): DireccionClienteDTO + actualizarDireccionCliente(idCliente: int, idDireccionCliente: int, datos: DireccionClienteDTO): DireccionClienteDTO + agregarAlCarrito(idCliente: int, idVariacion: int, cantidad: int): void + contarItemsCarrito(idCliente: int): int + publicarPromocion(datos: PromocionDTO): int + publicarNoticia(datos: NoticiaDTO): int	
«service» PortalPublicoService Implementa PortalPublicoContract y delega cada operación en el servicio del caso de uso (fachada).	Aplicación	- busqueda: BusquedaService - ficha: FichaProductoService - micrositio: MicrositioService - portada: PortadaService - contenido: ContenidoPortadaService - clientes: ClienteService - carrito: CarritoPortalService	Las 19 operaciones de PortalPublicoContract, con la misma firma.	Realiza PortalPublicoContract. Conoce a BusquedaService (1 a 1). Conoce a FichaProductoService (1 a 1). Conoce a MicrositioService (1 a 1). Conoce a PortadaService (1 a 1). Conoce a ContenidoPortadaService (1 a 1). Conoce a ClienteService (1 a 1). Conoce a CarritoPortalService (1 a 1).

«service» BusquedaService Control de CU-002 y CU-004: valida el criterio (RN-M1-08), pone el nombre del comercio y descarta como salvaguarda los que no estén Activos (RN-M1-01, RN-M1-02), combina la disponibilidad de M4, registra la búsqueda y ordena el menú de categorías según categorias_portal (CategoriaPortal::obtenerVisibles).	Aplicación	- catalogo: CatalogoClient - comercios: ComercioClient - inventario: InventarioClient	+ buscar(criterio: CriterioBusquedaDTO): ResultadoBusquedaDTO + listarCategoriasPortal(): CategoriaDTO[] - validarCriterio(criterio: CriterioBusquedaDTO): void - completarDatosComercio(productos: ProductoResumenDTO[], comercios: ComercioDTO[]): ProductoResumenDTO[] - combinarDisponibilidad(productos: ProductoResumenDTO[], disponibilidad: DisponibilidadDTO[]): ProductoResumenDTO[] - registrarBusqueda(criterio: CriterioBusquedaDTO, totalResultados: int): void - ordenarCategorias(categorias: CategoriaDTO[], configuracion: CategoriaPortal[]): CategoriaDTO[]	Conoce a CatalogoClient (1 a 1). Conoce a ComercioClient (1 a 1). Conoce a InventarioClient (1 a 1). Conoce a BusquedaUsuario (1 a 0..*). Conoce a CategoriaPortal (1 a 0..*).
«service» FichaProductoService Control de CU-003: arma la ficha con datos de M3, M2 y M4 (RN-M1-02, RN-M1-03). Una variación es comprable solo si M3 tiene la venta habilitada y M4 informa unidades (RN-M1-04).	Aplicación	- catalogo: CatalogoClient - comercios: ComercioClient - inventario: InventarioClient	+ obtenerFicha(idProducto: int): FichaProductoDTO - verificarComercioHabilitado(comercio: ComercioDTO): void - completarFicha(ficha: FichaProductoDTO, comercio: ComercioDTO, disponibilidad: DisponibilidadDTO[]): void - evaluarCompra(variacion: VariacionDTO): bool	Conoce a CatalogoClient (1 a 1). Conoce a ComercioClient (1 a 1). Conoce a InventarioClient (1 a 1).
«service» CarritoPortalService Botón Agregar al carrito de la ficha y contador del carrito del encabezado; ambos se resuelven en M5.	Aplicación	- carrito: CarritoClient	+ agregar(idCliente: int, idVariacion: int, cantidad: int): void + contarItems(idCliente: int): int	Conoce a CarritoClient (1 a 1).
«service» MicrositioService Control de CU-001 y CU-009: solo muestra	Aplicación	- comercios: ComercioClient - catalogo: CatalogoClient	+ listarComercios(pagina: int, porPagina: int): ListadoComerciosDTO	Conoce a CatalogoClient (1 a 1). Conoce a ComercioClient (1

comercios en estado Activo (RN-M1-01).		- inventario: InventarioClient	+ obtenerMicrositio(idComercio: int, pagina: int, porPagina: int): MicrositioDTO - verificarComercioHabilitado(comercio: ComercioDTO): void	a 1). Conoce a InventarioClient (1 a 1).
«service» PortadaService Control de CU-005 a CU-008: lee la selección de destacados de M1, pide los datos a M3 (filtro ids), M2 (lote) y M4, y si M2 o M3 no responden muestra la portada con aviso (RNF-M1-05).	Aplicación	- catalogo: CatalogoClient - comercios: ComercioClient - inventario: InventarioClient - contenido: ContenidoPortadaService	+ obtenerPortada(): PortadaDTO + listarDestacados(limite: int): DestacadosDTO - idsProductosDestacados(fecha: DateTime, limite: int): int[] - idsComerciosDestacados(fecha: DateTime, limite: int): int[]	Conoce a CatalogoClient (1 a 1). Conoce a ComercioClient (1 a 1). Conoce a InventarioClient (1 a 1). Conoce a ContenidoPortadaService (1 a 1). Conoce a ProductoDestacado (1 a 0..*). Conoce a ComercioDestacado (1 a 0..*).
«service» ContenidoPortadaService Promociones y noticias propias de M1: promociones vigentes y activas (RN-M1-10) y noticias publicadas en orden cronológico descendente (RF-005).	Aplicación	- bitacora: BitacoraAuditoria	+ listarPromocionesVigentes(fecha: DateTime): PromocionDTO[] + listarNoticiasPublicadas(): NoticiaDTO[] + publicarPromocion(datos: PromocionDTO): int + publicarNoticia(datos: NoticiaDTO): int	Conoce a BitacoraAuditoria (1 a 1). Conoce a PromocionPortal (1 a 0..*). Conoce a NoticiaPortal (1 a 0..*).
«service» ClienteService Control de CU-010 y de las direcciones: correo único (RN-M1-14); al registrar pide la cuenta al núcleo (ServicioIdentidad::crearCuenta) y crea el cliente y su dirección principal en una misma transacción; registra en bitácora.	Aplicación	- identidad: ServicioIdentidad - bitacora: BitacoraAuditoria	+ registrar(datos: ClienteDTO, contrasena: string): int + obtener(idCliente: int): ClienteDTO + actualizar(idCliente: int, datos: ClienteDTO): int + obtenerIdPorUsuario(idUsuario: int): int + listarDirecciones(idCliente: int): DireccionClienteDTO[] + obtenerDireccion(idCliente: int, idDireccionCliente: int): DireccionClienteDTO + agregarDireccion(idCliente: int, datos: DireccionClienteDTO): DireccionClienteDTO	Conoce a BitacoraAuditoria (1 a 1). Conoce a ServicioIdentidad (1 a 1). Conoce a Cliente (1 a 0..*).

			+ actualizarDireccion(idCliente: int, idDireccionCliente: int, datos: DireccionClienteDTO): DireccionClienteDTO - verificarCorreoDisponible(correo: string, idCliente: ?int): void	
«entity» Cliente Tabla clientes (entidad Cliente del modelo canónico). id_usuario enlaza con la cuenta del núcleo; la contraseña la guarda el núcleo.	Dominio	- id_cliente: int - id_usuario: int - nombre: string - apellidos: string - correo_electronico: string - telefono: ?string - cedula: string - estado_cliente: string	+ direccionPrincipal(): ?DireccionCliente + aDTO(): ClienteDTO	Composición con DireccionCliente (1 a 0..*). Extiende Model de Laravel.
«entity» DireccionCliente Tabla direcciones_cliente: direcciones de envío; una sola principal por cliente.	Dominio	- id_direccion_cliente: int - id_cliente: int - alias_direccion: string - nombre_destinatario: string - telefono_destinatario: string - provincia: string - canton: string - distrito: string - otras_senas: string - es_principal: bool	+ aDTO(): DireccionClienteDTO	Extiende Model de Laravel.
«entity» CategoriaPortal Tabla categorias_portal: orden y visibilidad de las categorías de M3 en el menú.	Dominio	- id_categoria_portal: int - id_categoria: int - orden_visualizacion: int - icono_url: ?string - visible_en_menu: bool	+ obtenerVisibles(): CategoriaPortal[]	La usa BusquedaService. Extiende Model de Laravel.

«entity» ProductoDestacado Tabla productos_destacados: selección de productos que M1 muestra en la portada (RN-M1-11).	Dominio	- id_producto_destacado: int - id_producto: int - posicion: int - fecha_inicio: DateTime - fecha_fin: DateTime - activo: bool	+ estaVigente(fecha: DateTime): bool	La usa PortadaService. Extiende Model de Laravel.
«entity» ComercioDestacado Tabla comercios_destacados: selección de comercios que M1 muestra en la portada (RN-M1-11).	Dominio	- id_comercio_destacado: int - id_comercio: int - posicion: int - fecha_inicio: DateTime - fecha_fin: DateTime - activo: bool	+ estaVigente(fecha: DateTime): bool	La usa PortadaService. Extiende Model de Laravel.
«entity» BusquedaUsuario Tabla busquedas_usuario: registro de cada búsqueda.	Dominio	- id_busqueda_usuario: int - id_cliente: ?int - identificador_sesion: ?string - termino_busqueda: string - cantidad_resultados: int - fecha_busqueda: DateTime	(sin métodos propios)	Composición con FiltroAplicado (1 a 0..*). Extiende Model de Laravel.
«entity» FiltroAplicado Tabla filtros_aplicados: filtros usados en cada búsqueda.	Dominio	- id_filtro_aplicado: int - id_busqueda_usuario: int - tipo_filtro: string - valor_filtro: string	(sin métodos propios)	Extiende Model de Laravel.

«entity» PromocionPortal Tabla promociones_portal: piezas del carrusel que dirigen a un producto, comercio, categoría o enlace.	Dominio	- id_promocion_portal: int - titulo_promocion: string - descripcion_promocion: ?string - imagen_url: string - tipo_destino: string - id_producto: ?int - id_comercio: ?int - id_categoria: ?int - enlace_url: ?string - orden_visualizacion: int - fecha_inicio: DateTime - fecha_fin: DateTime - activa: bool	+ estaVigente(fecha: DateTime): bool + aDTO(): PromocionDTO	La usa ContenidoPortadaService. Extiende Model de Laravel.
«entity» NoticiaPortal Tabla noticias_portal: noticias o avisos de la administración (borrador, publicada o archivada).	Dominio	- id_noticia_portal: int - id_usuario_autor: int - titulo_noticia: string - slug_noticia: string - resumen_noticia: string - contenido_noticia: string - imagen_url: ?string - estado_noticia: string - fecha_publicacion: ?DateTime	+ estaPublicada(): bool + aDTO(): NoticiaDTO	La usa ContenidoPortadaService. Extiende Model de Laravel.

«adapter» CatalogoClient Adaptador del contrato de M3 (buscarProductos, listarCategorias, listarMarcas, obtenerProducto, listarRelacionados); traduce a DTO de M1, guarda la lista de marcas en la caché y corta la espera a los 3 s (RNF-M1-04).	Infraestructura	- contrato: CatalogoServiceInterface - cache: CachePortal - tiempoMaximo: int = 3	+ buscarProductos(criterio: CriterioBusquedaDTO): ResultadoBusquedaDTO + buscarPorIds(ids: int[]): ProductoResumenDTO[] + listarCategorias(): CategoriaDTO[] + obtenerProducto(idProducto: int): FichaProductoDTO + listarRelacionados(idProducto: int): ProductoResumenDTO[] - nombreMarca(idMarca: ?int): ?string	Conoce a CatalogoServiceInterface (1 a 1). Conoce a CachePortal (1 a 1).
«adapter» ComercioClient Adaptador del contrato de M2 (ObtenerDatosComercioPublico, ObtenerResumenComerciosLote, ListarComerciosHabilitados); espera máxima 3 s (RNF-M1-04).	Infraestructura	- contrato: ComerciosServiceInterface - cache: CachePortal - tiempoMaximo: int = 3	+ obtenerDatosComercioPublico(idComercio: int): ComercioDTO + obtenerResumenComercios(ids: int[]): ComercioDTO[] + listarComerciosHabilitados(pagina: int, porPagina: int): ListadoComerciosDTO	Conoce a ComerciosServiceInterface (1 a 1). Conoce a CachePortal (1 a 1).
«adapter» InventarioClient Adaptador del contrato de M4 (consultarDisponibilidadProductoLote y consultarDisponibilidadLote); espera máxima 3 s.	Infraestructura	- contrato: InventarioModuleInterface - cache: CachePortal - tiempoMaximo: int = 3	+ consultarDisponibilidadProductos(idsProducto: int[]): DisponibilidadDTO[] + consultarDisponibilidadVariaciones(idsVariacion: int[]): DisponibilidadDTO[]	Conoce a InventarioModuleInterface (1 a 1). Conoce a CachePortal (1 a 1).
«adapter» CarritoClient Adaptador del contrato de M5 (agregarItemCarrito y obtenerCarrito).	Infraestructura	- contrato: CarritoServiceInterface - tiempoMaximo: int = 3	+ agregarItem(idCliente: int, idVariacion: int, cantidad: int): void + contarItems(idCliente: int): int	Conoce a CarritoServiceInterface (1 a 1).
«cache» CachePortal Caché de las respuestas de M2, M3 y M4 con vida máxima de 5 s (RNF-M1-06), sobre Cache de Laravel.	Infraestructura	- tiempoVida: int = 5	+ obtener(clave: string): mixed + guardar(clave: string, valor: mixed): void + invalidarComercio(idComercio: int): void + marcarAgotado(idProducto: int, idVariacion: ?int): void + quitarAgotado(idProducto: int, idVariacion: ?int): void	La usa CatalogoClient, ComercioClient, EventosComercioListener, EventosExistenciaListener, InventarioClient.

«listener» EventosComercioListener Consume los eventos de comercio de M2 e invalida la caché del comercio.	Infraestructura	- cache: CachePortal	+ alAprobarComercio(evento: ComercioAprobado): void + alSuspenderComercio(evento: ComercioSuspendido): void + alReactivarComercio(evento: ComercioReactivado): void	Conoce a CachePortal (1 a 1). Escucha ComercioAprobado. Escucha ComercioSuspendido. Escucha ComercioReactivado.
«listener» EventosExistenciaListener Consume los eventos de existencias de M4 y marca o quita el Agotado en la caché.	Infraestructura	- cache: CachePortal	+ alAgotarExistencia(evento: ExistenciaAgotada): void + alReponerExistencia(evento: ExistenciaRepuesta): void	Conoce a CachePortal (1 a 1). Escucha ExistenciaAgotada. Escucha ExistenciaRepuesta.
«DTO» CriterioBusquedaDTO Criterio de búsqueda con los filtros del contrato v1.1.0 (orden: precio_asc, precio_desc o novedad).	Aplicación	+ termino: ?string + idCategoria: ?int + idComercio: ?int + idMarca: ?int + precioMinimo: ?decimal + precioMaximo: ?decimal + orden: ?string + pagina: int = 1 + porPagina: int = 20	(sin métodos)	Viaja como parámetro o valor de retorno (ver firmas).
«DTO» ResultadoBusquedaDTO Resultado paginado de la búsqueda (también los productos del micrositio).	Aplicación	+ productos: ProductoResumenDTO[] + totalResultados: int + pagina: int + porPagina: int	(sin métodos)	Composición con ProductoResumenDTO (1 a 0..*).
«DTO» ProductoResumenDTO Producto en una lista (búsqueda, destacados, micrositio, relacionados).	Aplicación	+ idProducto: int + idComercio: int + nombre: string + imagenPrincipal: ?string + precio: decimal + precioHasta: ?decimal + moneda: string	(sin métodos)	Conoce a Disponibilidad (0..* a 1).

		+ nombreComercio: string + disponibilidad: Disponibilidad		
«DTO» CategoriaDTO Categoría de M3 con el orden del menú de M1.	Aplicación	+ idCategoria: int + nombre: string + idCategoriaPadre: ?int + iconoUrl: ?string + ordenVisualizacion: ?int	(sin métodos)	Viaja como parámetro o valor de retorno (ver firmas).
«DTO» FichaProductoDTO Ficha completa de un producto; compraHabilitada es verdadero si alguna variación es comprable.	Aplicación	+ idProducto: int + nombre: string + descripcion: ?string + marca: ?string + videoUrl: ?string + moneda: string + imagenes: string[] + variaciones: VariacionDTO[] + comercio: ComercioDTO + relacionados: ProductoResumenDTO[] + compraHabilitada: bool	(sin métodos)	Composición con VariacionDTO (1 a 1..*). Conoce a ComercioDTO (0..* a 1). Agregación de ProductoResumenDTO (0..* a 0..*).
«DTO» VariacionDTO Variación de un producto; ventaHabilitada viene de M3 y disponibilidad de M4.	Aplicación	+ idVariacion: int + sku: string + atributos: array + precio: decimal + imagenes: string[] + ventaHabilitada: bool + disponibilidad: Disponibilidad + comprable: bool	(sin métodos)	Conoce a Disponibilidad (0..* a 1).

«DTO» ComercioDTO Datos públicos de un comercio que entrega M2.	Aplicación	+ idComercio: int + nombreComercial: string + logotipo: ?string + descripcion: ?string + imagenes: string[] + horarios: array + contacto: array + redesSociales: array + politicas: array + estado: EstadoComercio	(sin métodos)	Conoce a EstadoComercio (0..* a 1).
«DTO» ListadoComerciosDTO Página de comercios habilitados.	Aplicación	+ comercios: ComercioDTO[] + total: int + pagina: int + porPagina: int	(sin métodos)	Agregación de ComercioDTO (0..* a 0..*).
«DTO» DisponibilidadDTO Estado de disponibilidad que entrega M4.	Aplicación	+ idProducto: int + idVariacion: ?int + estado: Disponibilidad	(sin métodos)	Conoce a Disponibilidad (0..* a 1).
«DTO» DestacadosDTO Productos y comercios destacados.	Aplicación	+ productos: ProductoResumenDTO[] + comercios: ComercioDTO[] + seccionesNoDisponibles: string[]	(sin métodos)	Agregación de ProductoResumenDTO (0..* a 0..*). Agregación de ComercioDTO (0..* a 0..*).
«DTO» PortadaDTO Contenido de la página principal.	Aplicación	+ destacados: ?DestacadosDTO + promociones: PromocionDTO[] + noticias: NoticiaDTO[] + avisos: string[]	(sin métodos)	Composición con DestacadosDTO (1 a 0..1). Composición con PromocionDTO (1 a 0..*). Composición con NoticiaDTO (1 a 0..*).
«DTO» MicrositioDTO Micrositio público de un comercio.	Aplicación	+ comercio: ComercioDTO	(sin métodos)	Conoce a ComercioDTO (0..* a 1). Composición

		+ productos: ResultadoBusquedaDTO		con ResultadoBusquedaDTO (1 a 1).
«DTO» ClienteDTO Datos del cliente que se exponen; direccionEnvio es la dirección principal en texto.	Aplicación	+ idCliente: ?int + nombre: string + apellidos: string + correoElectronico: string + telefono: ?string + cedula: string + direccionEnvio: string	(sin métodos)	Viaja como parámetro o valor de retorno (ver firmas).
«DTO» DireccionClienteDTO Dirección de envío del cliente.	Aplicación	+ idDireccionCliente: ?int + idCliente: ?int + aliasDireccion: string + nombreDestinatario: string + telefonoDestinatario: string + provincia: string + canton: string + distrito: string + otrasSenas: string + esPrincipal: bool	(sin métodos)	Viaja como parámetro o valor de retorno (ver firmas).
«DTO» PromocionDTO Promoción para mostrar o publicar; idDestino es el producto, comercio o categoría según tipoDestino.	Aplicación	+ idPromocion: ?int + titulo: string + descripcion: ?string + imagenUrl: string + tipoDestino: string + idDestino: ?int + enlaceUrl: ?string + orden: int + fechaInicio: DateTime + fechaFin: DateTime	(sin métodos)	Viaja como parámetro o valor de retorno (ver firmas).

«DTO» NoticiaDTO Noticia para mostrar o publicar.	Aplicación	+ idNoticia: ?int + titulo: string + slug: string + resumen: string + contenido: string + imagenUrl: ?string + estado: string + fechaPublicacion: ?DateTime	(sin métodos)	Viaja como parámetro o valor de retorno (ver firmas).
«enumeration» Disponibilidad **«enumeration» EstadoComercio**	Aplicación	Disponibilidad: Disponible, Agotado EstadoComercio: Pendiente, Activo, Rechazado, Suspendido (valores de M2)	(sin métodos)	Las usan ProductoResumenDTO, VariacionDTO, DisponibilidadDTO y ComercioDTO.
«exception» PortalPublicoException Base de las excepciones de M1; expone el código de error que el manejador del núcleo pone en la respuesta.	Aplicación	# codigo: string	+ codigo(): string	Hereda de RuntimeException (PHP).
«exception» ClienteNoEncontradoException Cliente inexistente.	Aplicación	# codigo: string = "ERR-004"	(heredados)	Hereda de PortalPublicoException.
«exception» DireccionNoEncontradaException Dirección inexistente o de otro cliente.	Aplicación	# codigo: string = "ERR-004"	(heredados)	Hereda de PortalPublicoException.
«exception» RecursoNoEncontradoException Producto o comercio inexistente, no publicado o no activo.	Aplicación	# codigo: string = "ERR-004"	(heredados)	Hereda de PortalPublicoException.
«exception» CriterioInvalidoException Criterio de búsqueda inválido (RN-M1-08).	Aplicación	# codigo: string = "ERR-005"	(heredados)	Hereda de PortalPublicoException.

«exception» DatosInvalidosException Datos obligatorios faltantes o con formato incorrecto.	Aplicación	# codigo: string = "ERR-005" - detalles: array	(heredados)	Hereda de PortalPublicoException.
«exception» DependenciaNoDisponibleException M2, M3, M4 o M5 no respondieron en 3 s (RNF-M1-04).	Aplicación	# codigo: string = "ERR-006"	(heredados)	Hereda de PortalPublicoException.
«exception» CorreoYaRegistradoException Correo ya registrado (RN-M1-14).	Aplicación	# codigo: string = "ERR-M1-001"	(heredados)	Hereda de PortalPublicoException.

3.2 Clases del núcleo y de otros módulos que se usan

Los nombres de las interfaces de M2, M3 y M4 y de las clases del núcleo son los registrados en el documento colectivo del Comité. Las firmas de ServicioIdentidad::crearCuenta y de BitacoraAuditoria::registrar, y el nombre de la interfaz de M5, se toman como supuesto del subgrupo hasta confirmarlos con la sección 5 del documento colectivo y con el contrato de M5; si cambian, solo se ajusta la firma en este diagrama. En la vista 2 cada interfaz externa muestra solo las operaciones que usa M1, con la firma con que M1 las invoca; devuelven arreglos con los datos del otro módulo, que el adaptador correspondiente traduce a DTO de M1.

Clase o interfaz externa	Origen	Para qué la usa el módulo
«interface» CatalogoServiceInterface	Contrato de M3 (v1.2.1)	CatalogoClient: buscarProductos (también con filtro de ids para destacados y de id_comercio para el micrositio), listarCategorias, listarMarcas, obtenerProducto (con la venta habilitada de cada variación) y listarRelacionados.
«interface» ComerciosServiceInterface	Contrato de M2 (v1.2.1)	ComercioClient: ObtenerDatosComercioPublico (micrositio), ObtenerResumenComerciosLote (nombre y estado en resultados, ficha y destacados) y ListarComerciosHabilitados. Los estados son Pendiente, Activo, Rechazado y Suspendido.
«interface» InventarioModuleInterface	Contrato de M4 (v1.1.0)	InventarioClient: consultarDisponibilidadProductoLote (por producto) y consultarDisponibilidadLote (por variación).
«interface» CarritoServiceInterface	Contrato de M5	CarritoClient: agregarItemCarrito (botón Agregar al carrito de la ficha) y obtenerCarrito (contador del encabezado).
«event» ComercioAprobado, ComercioSuspendido, ComercioReactivado	Eventos de M2 (v1)	EventosComercioListener invalida en CachePortal los datos del comercio.

«event» ExistenciaAgotada, ExistenciaRepuesta	Eventos de M4 (1.0.0)	EventosExistenciaListener marca o quita el Agotado del producto o la variación en CachePortal.
«núcleo» ServicioIdentidad	Núcleo (5.1)	ClienteService pide crearCuenta al registrar un cliente; M1 guarda solo el id_usuario y nunca la contraseña.
«núcleo» Usuario	Núcleo (5.1)	Usuario de la sesión que recibe ClientePolicy (RN-M1-16). El middleware EstablecerCliente del núcleo obtiene el id_cliente con PortalPublicoContract::obtenerIdClientePorUsuario.
«núcleo» BitacoraAuditoria	Núcleo (5.4)	Registrar quién crea o modifica un cliente, una dirección, una promoción o una noticia (auditoría).
Manejador de errores del núcleo	Núcleo (5.5)	Convierte cada PortalPublicoException en {"error": {codigo, mensaje, detalles}} usando codigo() (ERR-004, ERR-005, ERR-006 y ERR-M1-001).
«PHP» RuntimeException	PHP	Clase base de PortalPublicoException, como define el contrato v1.1.0.
Request, JsonResponse, RedirectResponse, Inertia\Response, Model, Cache	Laravel e Inertia	Entrada y salida de los controladores, clase base de los modelos Eloquent y almacén de CachePortal.

4. Diagramas de interacción actualizados

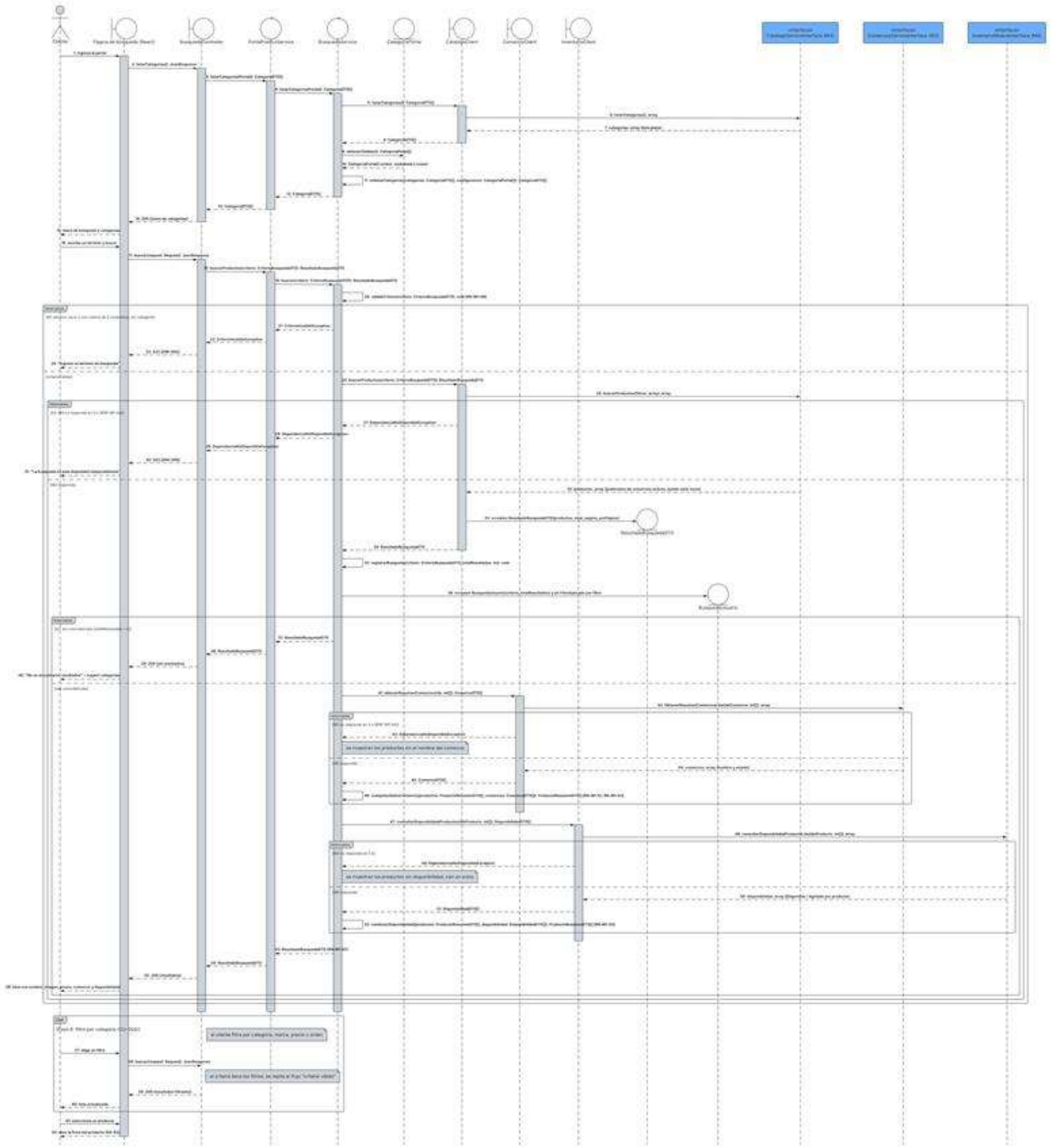
Diagramas de secuencia de los casos de uso del PC2 actualizados con las clases de diseño, incluyendo las llamadas a otros módulos.

Se presentan los tres casos de uso especificados en el PC2: CU-002 Buscar productos (DS-01), CU-003 Consultar ficha de producto (DS-02) y CU-009 Consultar información pública de un comercio (DS-03), con las clases de diseño de la sección 4. Cada mensaje entre clases del módulo corresponde a un método del diagrama de clases, con su firma. Las llamadas a M2, M3 y M4 pasan por los adaptadores CatalogoClient, ComercioClient e InventarioClient y llegan a la interfaz publicada de cada módulo, que aparece como participante «interface» (recuadro azul, por ejemplo, CatalogoServiceInterface (M3)), con la firma de la operación de su contrato; nunca a sus tablas. Se usan los mismos íconos que en la sección 3: límite para la página React, los controladores y los adaptadores; control para los servicios; entidad para los DTO y modelos que se crean.

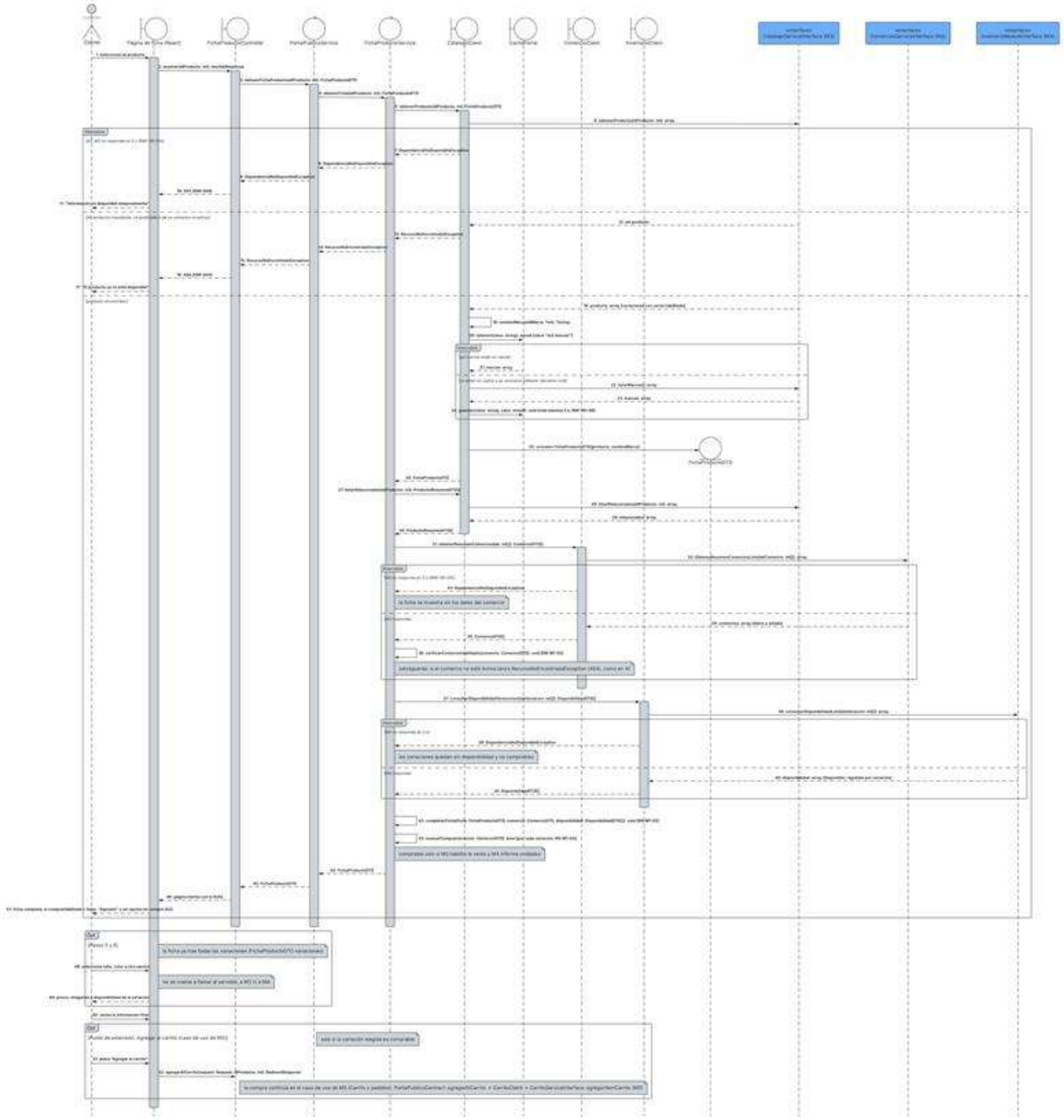
- **Flujos de error:** las excepciones del contrato (subclases de PortalPublicoException) las convierte el manejador de errores del núcleo en la estructura estándar: 422 con ERR-005, 404 con ERR-004 y 503 con ERR-006.
- **Fallas parciales (RNF-M1-04 y RNF-M1-05):** si M2 o M4 no responden en 3 s, los resultados se muestran sin el dato que falta (nombre del comercio o disponibilidad) y con un aviso; solo la falla de M3, o la de M2 en el micrositio, corta el caso de uso.
- **Llamadas al núcleo:** los tres casos de uso son públicos (RN-M1-13), así que no requieren autenticación ni verificación de permisos, y no registran bitácora porque no modifican datos de otros. La búsqueda se guarda en las tablas propias busquedas_usuario y filtros_aplicados.
- **Caché:** CachePortal guarda las respuestas de M2, M3 y M4 por 5 s como máximo (RNF-M1-06). En DS-02 se dibuja para la lista de marcas de M3 (obtener y guardar), que se usa en cada ficha; en los demás mensajes no se repite para no recargar los diagramas.
- **Creación de objetos:** los mensajes «create» llegan a la cabecera del objeto creado, que empieza su línea de vida en ese punto (ResultadoBusquedaDTO y BusquedaUsuario en DS-01, FichaProductoDTO en DS-02 y MicrositioDTO en DS-03).
- **Datos propios de M1:** DS-01 lee la configuración del menú con CategoriaPortal::obtenerVisibles (orden, visibilidad e ícono de categorias_portal) antes de ordenar las categorías que entrega M3.
- **Punto de extensión:** DS-02 termina con el botón Agregar al carrito, que continúa en el caso de uso de M5 (PortalPublicoContract::agregarAlCarrito → CarritoClient → CarritoServiceInterface::agregarItemCarrito).
- **Flujos alternativos:** se muestran como fragmentos alt con el mismo nombre del PC2 (A1, A2, A3, A4); los pasos opcionales, como fragmentos opt. Las variantes que solo cambian lo que muestra la página (A2 en DS-02 y DS-03) se indican en el último mensaje hacia el cliente.

Diagrama	Caso de uso	Módulos externos llamados	Operación o evento del contrato usado
DS-01	CU-002 Buscar productos	M3 Catálogo de productos M2 Micrositios y administración de comercios	M3: listarCategorias y buscarProductos (v1.2.1) M2: ObtenerResumenComerciosLote (v1.2.1)

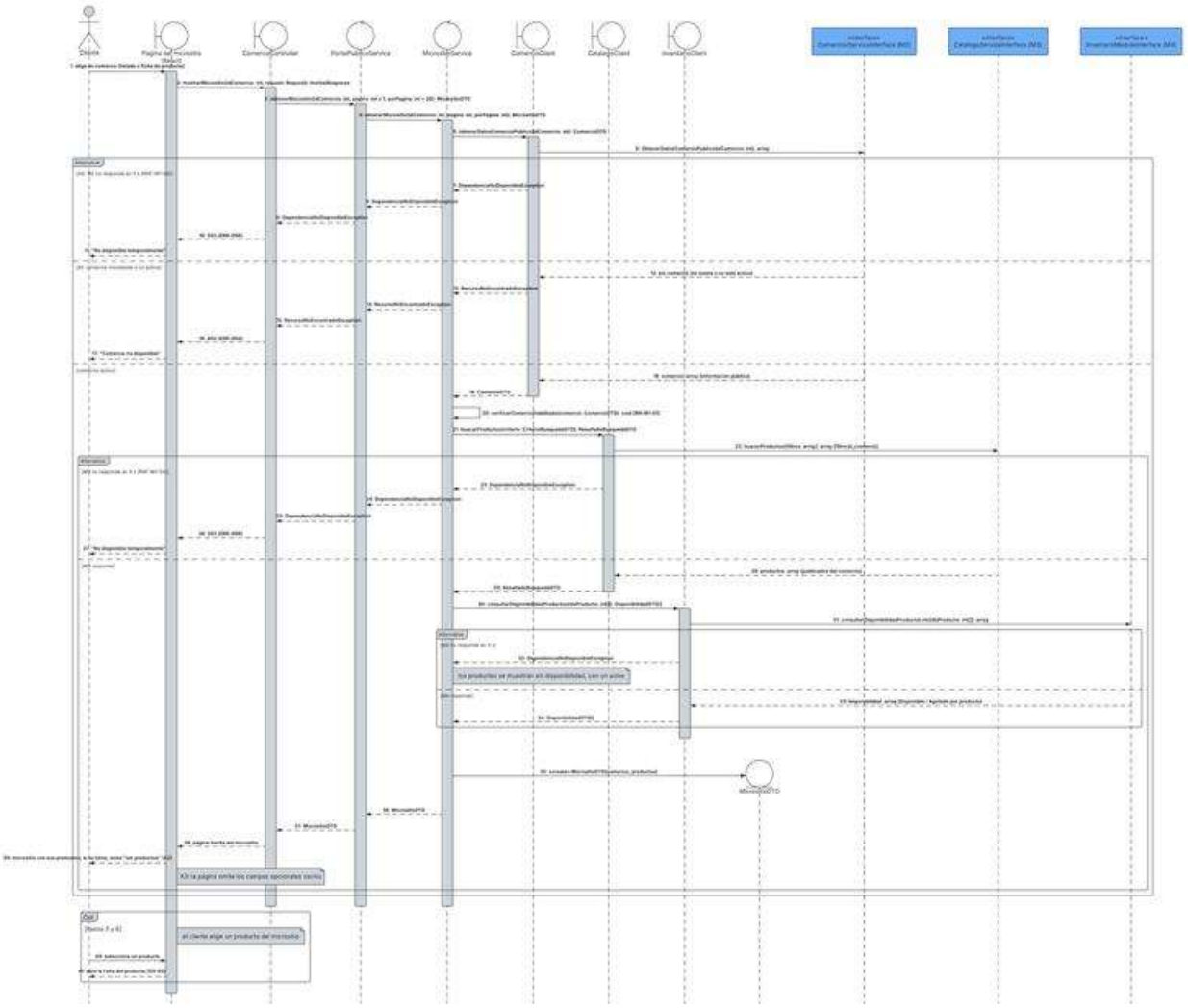
		M4 Inventario multiempresa	M4: consultarDisponibilidadProductoLote (v1.1.0)
DS-02	CU-003 Consultar ficha de producto	M3 Catálogo de productos M2 Micrositios y administración de comercios M4 Inventario multiempresa M5 Carrito y pedidos	M3: obtenerProducto, listarMarcas y listarRelacionados (v1.2.1) M2: ObtenerResumenComerciosLote (v1.2.1) M4: consultarDisponibilidadLote (v1.1.0) M5 (punto de extensión): agregarItemCarrito
DS-03	CU-009 Consultar información pública de un comercio	M2 Micrositios y administración de comercios M3 Catálogo de productos M4 Inventario multiempresa	M2: ObtenerDatosComercioPublico (v1.2.1) M3: buscarProductos con filtro id_comercio (v1.2.1) M4: consultarDisponibilidadProductoLote (v1.1.0)



DS-01. Diagrama de secuencia de CU-002 Buscar productos con las clases de diseño.



DS-02. Diagrama de secuencia de CU-003 Consultar ficha de producto con las clases de diseño.



DS-03. Diagrama de secuencia de CU-009 Consultar información pública de un comercio con las clases de diseño.

5. Modelo de datos del módulo

Modelo en tercera forma normal, implementado como migraciones de Laravel, con restricciones de unicidad y verificación que implementan las reglas de negocio. Se entrega además el SQL resultante.

5.1 Tablas del módulo

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
Clientes	id_cliente	id_usuario	Entidad canónica Cliente (propietario de M1, según PC2). Guarda los datos de la persona compradora
direcciones_clientes	id_direccion_cliente	id_cliente	Direcciones de envío de cada cliente. Se separan de clientes porque un cliente puede registrar varias (1FN). M5 las consulta por contrato para calcular y registrar el envío.
categorias_portal	id_categoria_portal	id_categoria	Qué categorías del catálogo (M3) se muestran en el menú del portal y en qué orden. La categoría y sus subcategorías pertenecen a M3; M1 solo guarda su presentación.
productos_destacados	id_producto_destacado	id_producto	Productos del catálogo (M3) que el portal presenta de forma preferente en la página principal durante un periodo.
promociones_portal	id_promocion_portal	- id_producto - id_comercio - id_categoria	Piezas promocionales que se muestran en el portal (carrusel o bloques de la página principal) y que dirigen a un producto, comercio, categoría o enlace. No calcula descuentos: los cupones y precios son de M5 y M3.

Tabla	Clave primaria	Claves compartidas que guarda	Descripción
noticias_portal	id_noticia_portal	id:_usuario_autor	Noticias o avisos publicados en el portal por la administración de la plataforma.
comercios_destacados	id_comercio_destacado	id_comercio	Comercios (M2) que el portal presenta de forma preferente en la página principal durante un periodo.
Busqueda_usuario	id_busquedas_usuario	id_cliente	Registro de cada búsqueda realizada en el portal. Permite ofrecer a M3 los términos más buscados, según la matriz de dependencias del PC2.
filtros_aplicados	id_filtro_aplicado	id_busqueda_usuario	Filtros usados en cada búsqueda. Van en tabla aparte porque una búsqueda puede tener varios filtros (1FN).

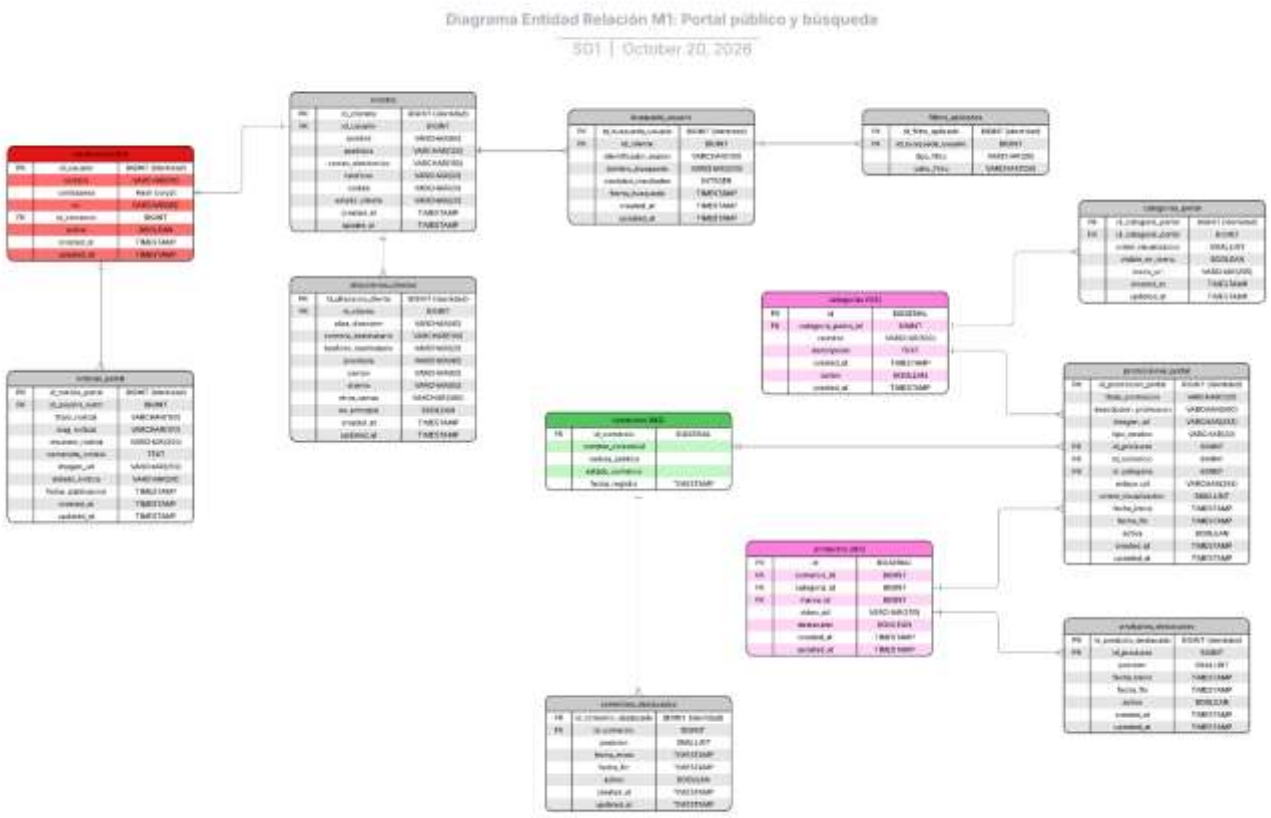


Diagrama entidad relación Modulo 1

5.2 Verificación de tercera forma normal

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
Clientes	Cumple	Cumple	Cumple	Atributos atómicos; clave primaria simple (id_cliente). Referencia al núcleo solo mediante clave compartida id_usuario (sin JOINs físicos).
direcciones_cliente	Cumple	Cumple	Cumple	Separado de clientes para cumplir 1FN ante direcciones múltiples; clave

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
				primaria simple y dependencia directa al cliente.
categorias_portal	Cumple	Cumple	Cumple	Datos de presentación visual desacoplados de M3; clave primaria simple.
promociones_portal	Cumple	Cumple	Cumple	Atributos atómicos y restricciones de coherencia por tipo de destino; clave primaria simple.
noticias_portal	Cumple	Cumple	Cumple	Contenido textual y metadatos con clave primaria simple (id_noticia_portal) y autoría en clave compartida.
busquedas_usuario	Cumple	Cumple	Cumple	Bitácora temporal con clave primaria simple e integridad referencial opcional hacia clientes.
filtros_aplicados	Cumple	Cumple	Cumple	Normalizado en tabla independiente (1FN) respecto a búsquedas múltiples; clave primaria simple.
Comercios_destacados	Cumple	Cumple	Cumple	Normalizado en tabla independiente (1FN) respecto a búsquedas múltiples; clave primaria simple.

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
Productos_destacados	Cumple	Cumple	Cumple	Datos de vitrina con clave primaria simple; el comercio se obtiene por contrato con M3 para evitar dependencias transitivas.

5.3 Restricciones que implementan reglas de negocio

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración
RN-M1-14 (Correo único de cliente)	clientes	UNIQUE	Correo electrónico único sin distinguir mayúsculas (LOWER sobre índice).	uq_clientes_correo_electronico
RN-M1-15 (Cédula fiscal obligatoria del cliente)	Clientes	UNIQUE	Cédula fiscal única requerida para emisión de facturación electrónica en M6 (LOWER sobre índice).	uq_clientes_cedula
RN-M1-10 (Vigencia de promociones)	promociones_portal	CHECK	fecha_fin > fecha_inicio.	chk_promociones_vigencia
RN-M1-05 (Vigencia de productos destacados)	productos_destacados	CHECK	fecha_fin > fecha_inicio.	chk_productos_destacados_vigencia
RN-M1-08 (Filtros únicos por búsqueda)	filtros_aplicados	UNIQUE	(id_busqueda_usuario, tipo_filtro, valor_filtro).	uq_busqueda_tipo_valor_filtro

Regla de negocio (PC2)	Tabla	Tipo de restricción	Definición	Nombre en la migración

5.4 Migraciones

Archivo de migración	Tabla	Descripción
database/migrations/2026_10_01_044846_create_clientes_table.php	clientes	Crea la entidad compradora, vinculación 1:1 con el núcleo (id_usuario), datos fiscales y de contacto.
database/migrations/2026_10_01_044901_create_direcciones_cliente_table.php	direcciones_cliente	Crea la tabla de direcciones de envío con control de alias y dirección principal única por cliente.
database/migrations/2026_10_01_044911_create_categorias_portal_table.php	categorias_portal	Configura el orden visual, visibilidad e íconos de las categorías obtenidas de M3.
database/migrations/2026_10_01_044920_create_productos_destacados_table.php	productos_destacados	Gestiona la vitrina de productos destacados en portada con control de rangos de fechas.

Archivo de migración	Tabla	Descripción
database/migrations/2026_10_01_045001_create_promociones_portal_table.php	promociones_portal	Crea la tabla de piezas promocionales y carruseles con validación de fechas y destinos.
database/migrations/2026_10_01_0450017_create_noticias_portal_table.php	noticias_portal	Administra la publicación de avisos, comunicados y metadatos de blog corporativo.
database/migrations/2026_10_01_045007_create_busquedas_usuario_table.php	busquedas_usuario	Registra el historial de consultas de búsqueda para analítica y tendencias del portal.
database/migrations/2026_10_01_0450023_create_filtros_aplicados_table.php	filtros_aplicados	Almacena los parámetros y filtros granulares utilizados en cada búsqueda de la bitácora.
database/migrations/2026_10_01_044955_create_comercios_destacados_table.php	Comercios_destacados	Gestiona la vitrina de comercios destacados en portada con control de rangos de fechas.

5.5 SQL resultante

```
-- ======================================================================
```

```
-- Pacíficos Online — M1 Portal público y búsqueda
-- Sección 6.5: SQL resultante del modelo de datos (PostgreSQL)

-- clientes
CREATE TABLE clientes (
   id_cliente         bigserial PRIMARY KEY,
   id_usuario         bigint NOT NULL,
   cedula             varchar(20) NOT NULL,
   nombre             varchar(80) NOT NULL,
   apellidos          varchar(120) NOT NULL,
   correo_electronico  varchar(150) NOT NULL,
   telefono           varchar(20),
   estado_cliente      varchar(20) NOT NULL DEFAULT 'activo',
   created_at         timestamp(0),
   updated_at         timestamp(0),
   CONSTRAINT uq_clientes_id_usuario UNIQUE (id_usuario),
   CONSTRAINT uq_clientes_cedula UNIQUE (cedula),
   CONSTRAINT chk_clientes_estado CHECK (estado_cliente IN ('activo', 'inactivo')),
   CONSTRAINT chk_clientes_nombre_no_vacio CHECK (btrim(nombre) <> '' AND btrim(apellidos) <> ''),
   CONSTRAINT chk_clientes_cedula_no_vacia CHECK (btrim(cedula) <> ''),
   CONSTRAINT chk_clientes_telefono CHECK (telefono IS NULL OR telefono ~ '^[0-9 +-]+$')
);
CREATE UNIQUE INDEX uq_clientes_correo_electronico ON clientes (LOWER(correo_electronico));

-- direcciones_cliente
CREATE TABLE direcciones_cliente (
   id_direccion_cliente  bigserial PRIMARY KEY,
   id_cliente           bigint NOT NULL,
   alias_direccion       varchar(40) NOT NULL,
   nombre_destinatario   varchar(150) NOT NULL,
   telefono_destinatario varchar(20) NOT NULL,
   provincia            varchar(40) NOT NULL,
   canton               varchar(60) NOT NULL,
   distrito             varchar(60) NOT NULL,
   otras_senas          varchar(300) NOT NULL,
   es_principal         boolean NOT NULL DEFAULT false,
   created_at           timestamp(0),
```

```
  updated_at          timestamp(0),
  CONSTRAINT direcciones_cliente_id_cliente_foreign FOREIGN KEY (id_cliente)
    REFERENCES clientes (id_cliente) ON DELETE CASCADE,
  CONSTRAINT uq_direcciones_cliente_alias UNIQUE (id_cliente, alias_direccion),
  CONSTRAINT chk_direcciones_cliente_no_vacios CHECK (btrim(nombre_destinatario) <> '' AND
btrim(otras_senas) <> ''),
  CONSTRAINT chk_direcciones_cliente_telefono CHECK (telefono_destinatario ~ '^[0-9 +-]+$')
);
CREATE UNIQUE INDEX uq_direcciones_cliente_principal ON direcciones_cliente (id_cliente) WHERE
es_principal;

-- categorias_portal
CREATE TABLE categorias_portal (
  id_categoria_portal  bigserial PRIMARY KEY,
  id_categoria        bigint NOT NULL,
  orden_visualizacion  smallint NOT NULL,
  visible_en_menu     boolean NOT NULL DEFAULT true,
  icono_url           varchar(255),
  created_at          timestamp(0),
  updated_at          timestamp(0),
  CONSTRAINT uq_categorias_portal_id_categoria UNIQUE (id_categoria),
  CONSTRAINT chk_categorias_portal_orden CHECK (orden_visualizacion > 0)
);

-- productos_destacados
CREATE TABLE productos_destacados (
  id_producto_destacado bigserial PRIMARY KEY,
  id_producto          bigint NOT NULL,
  posicion             smallint NOT NULL,
  fecha_inicio         timestamp(0) NOT NULL,
  fecha_fin            timestamp(0) NOT NULL,
  activo               boolean NOT NULL DEFAULT true,
  created_at           timestamp(0),
  updated_at           timestamp(0),
  CONSTRAINT chk_productos_destacados_posicion CHECK (posicion > 0),
  CONSTRAINT chk_productos_destacados_vigencia CHECK (fecha_fin > fecha_inicio)
);
```

```
-- comercios_destacados
CREATE TABLE comercios_destacados (
   id_comercio_destacado bigserial PRIMARY KEY,
   id_comercio          bigint NOT NULL,
   posicion             smallint NOT NULL,
   fecha_inicio         timestamp(0) NOT NULL,
   fecha_fin            timestamp(0) NOT NULL,
   activo               boolean NOT NULL DEFAULT true,
   created_at           timestamp(0),
   updated_at           timestamp(0),
   CONSTRAINT chk_comercios_destacados_posicion CHECK (posicion > 0),
   CONSTRAINT chk_comercios_destacados_vigencia CHECK (fecha_fin > fecha_inicio)
);

-- promociones_portal
CREATE TABLE promociones_portal (
   id_promocion_portal   bigserial PRIMARY KEY,
   titulo_promocion      varchar(120) NOT NULL,
   descripcion_promocion varchar(500),
   imagen_url            varchar(255) NOT NULL,
   tipo_destino          varchar(20) NOT NULL,
   id_producto           bigint,
   id_comercio           bigint,
   id_categoria          bigint,
   enlace_url            varchar(255),
   orden_visualizacion   smallint NOT NULL,
   fecha_inicio          timestamp(0) NOT NULL,
   fecha_fin             timestamp(0) NOT NULL,
   activa                boolean NOT NULL DEFAULT true,
   created_at            timestamp(0),
   updated_at            timestamp(0),
   CONSTRAINT chk_promociones_portal_tipo_destino CHECK (tipo_destino IN ('producto', 'comercio',
'categoria', 'enlace')),
   CONSTRAINT chk_promociones_portal_destino CHECK (
      (tipo_destino = 'producto'  AND id_producto  IS NOT NULL AND id_comercio IS NULL AND id_categoria IS
NULL AND enlace_url IS NULL) OR
```

```
      (tipo_destino = 'comercio'  AND id_comercio  IS NOT NULL AND id_producto IS NULL AND id_categoria IS NULL AND enlace_url IS NULL) OR
      (tipo_destino = 'categoria' AND id_categoria IS NOT NULL AND id_producto IS NULL AND id_comercio  IS NULL AND enlace_url IS NULL) OR
      (tipo_destino = 'enlace'    AND enlace_url   IS NOT NULL AND id_producto IS NULL AND id_comercio  IS NULL AND id_categoria IS NULL)
   ),
   CONSTRAINT chk_promociones_portal_vigencia CHECK (fecha_fin > fecha_inicio),
   CONSTRAINT chk_promociones_portal_orden CHECK (orden_visualizacion > 0),
   CONSTRAINT chk_promociones_portal_titulo CHECK (btrim(titulo_promocion) <> '')
);

-- noticias_portal
CREATE TABLE noticias_portal (
   id_noticia_portal  bigserial PRIMARY KEY,
   id_usuario_autor   bigint NOT NULL,
   titulo_noticia     varchar(150) NOT NULL,
   slug_noticia       varchar(170) NOT NULL,
   resumen_noticia    varchar(300) NOT NULL,
   contenido_noticia  text NOT NULL,
   imagen_url         varchar(255),
   estado_noticia     varchar(20) NOT NULL DEFAULT 'borrador',
   fecha_inicio       timestamp(0) NOT NULL,
   fecha_fin          timestamp(0) NOT NULL,
   created_at         timestamp(0),
   updated_at         timestamp(0),
   CONSTRAINT uq_noticias_portal_slug UNIQUE (slug_noticia),
   CONSTRAINT chk_noticias_portal_estado CHECK (estado_noticia IN ('borrador', 'publicada', 'archivada')),
   CONSTRAINT chk_noticias_portal_vigencia CHECK (fecha_fin > fecha_inicio),
   CONSTRAINT chk_noticias_portal_slug CHECK (slug_noticia ~ '^[a-z0-9]+(-[a-z0-9]+)*$'),
   CONSTRAINT chk_noticias_portal_titulo CHECK (btrim(titulo_noticia) <> '')
);

-- busquedas_usuario
CREATE TABLE busquedas_usuario (
   id_busqueda_usuario  bigserial PRIMARY KEY,
   id_cliente           bigint,
   identificador_sesion varchar(100),
```

```
    termino_busqueda     varchar(200) NOT NULL,
    cantidad_resultados  integer NOT NULL,
    fecha_busqueda       timestamp(0) NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT busquedas_usuario_id_cliente_foreign FOREIGN KEY (id_cliente)
        REFERENCES clientes (id_cliente) ON DELETE SET NULL,
    CONSTRAINT chk_busquedas_usuario_resultados CHECK (cantidad_resultados >= 0),
    CONSTRAINT chk_busquedas_usuario_termino CHECK (btrim(termino_busqueda) <> '')
);

-- filtros_aplicados
CREATE TABLE filtros_aplicados (
    id_filtro_aplicado   bigserial PRIMARY KEY,
    id_busqueda_usuario  bigint NOT NULL,
    tipo_filtro          varchar(30) NOT NULL,
    valor_filtro         varchar(100) NOT NULL,
    CONSTRAINT filtros_aplicados_id_busqueda_usuario_foreign FOREIGN KEY (id_busqueda_usuario)
        REFERENCES busquedas_usuario (id_busqueda_usuario) ON DELETE CASCADE,
    CONSTRAINT uq_filtros_aplicados_busqueda_tipo_valor UNIQUE (id_busqueda_usuario, tipo_filtro,
valor_filtro),
    CONSTRAINT chk_filtros_aplicados_tipo CHECK (tipo_filtro IN ('categoria', 'subcategoria', 'marca', 'comercio',
                                                 'precio_minimo', 'precio_maximo', 'disponibilidad', 'orden'))
);

-- ------------------------------------------------------------------
-- Parte 2: llaves foráneas hacia otros módulos (migración 100010)
-- Requiere que existan: usuarios (núcleo), comercios (M2),
-- productos y categorias (M3).
-- ------------------------------------------------------------------

ALTER TABLE clientes            ADD CONSTRAINT fk_clientes_usuarios            FOREIGN KEY (id_usuario)
REFERENCES usuarios (id);
ALTER TABLE noticias_portal     ADD CONSTRAINT fk_noticias_portal_usuarios     FOREIGN KEY
(id_usuario_autor) REFERENCES usuarios (id);
ALTER TABLE categorias_portal   ADD CONSTRAINT fk_categorias_portal_categorias  FOREIGN KEY
(id_categoria)    REFERENCES categorias (id_categoria);
ALTER TABLE productos_destacados ADD CONSTRAINT fk_productos_destacados_productos FOREIGN KEY
(id_producto)     REFERENCES productos (id_producto);
ALTER TABLE comercios_destacados ADD CONSTRAINT fk_comercios_destacados_comercios FOREIGN KEY
(id_comercio)     REFERENCES comercios (id_comercio);
```

```
ALTER TABLE promociones_portal   ADD CONSTRAINT fk_promociones_portal_productos   FOREIGN KEY (id_producto)      REFERENCES productos (id_producto);

ALTER TABLE promociones_portal   ADD CONSTRAINT fk_promociones_portal_comercios   FOREIGN KEY (id_comercio)      REFERENCES comercios (id_comercio);

ALTER TABLE promociones_portal   ADD CONSTRAINT fk_promociones_portal_categorias  FOREIGN KEY (id_categoria)     REFERENCES categorias (id_categoria);
```

6. Patrones de diseño aplicados (mínimo tres)

6.1 Facade

Categoría	Estructural
Problema del módulo que resuelve	Los controladores de M1 no deben conocer ni orquestar manualmente cada servicio de aplicación y cada adaptador (`CatalogoClient`, `ComercioClient`, `InventarioClient`, `CarritoClient`) para resolver una petición. El módulo necesita un único punto de entrada que agrupe esa complejidad detrás de un contrato estable.
Dónde se aplica	Clase `PortalPublicoService`, que implementa el contrato `PortalPublicoContract`
Justificación	Los controladores llaman un solo punto de entrada en vez de conocer y coordinar cada servicio y adaptador por separado; si se reorganiza un servicio interno, la fachada es el único lugar que cambia, y el contrato `PortalPublicoContract` se mantiene estable de cara a otros módulos
Alternativa descartada	Que cada controlador llame directamente a los servicios de aplicación y a los adaptadores. Se descartó porque dispersaría el conocimiento de la arquitectura interna del módulo en cada controlador y porque ya existe, según el diagrama real, una fachada pensada exactamente para esto.

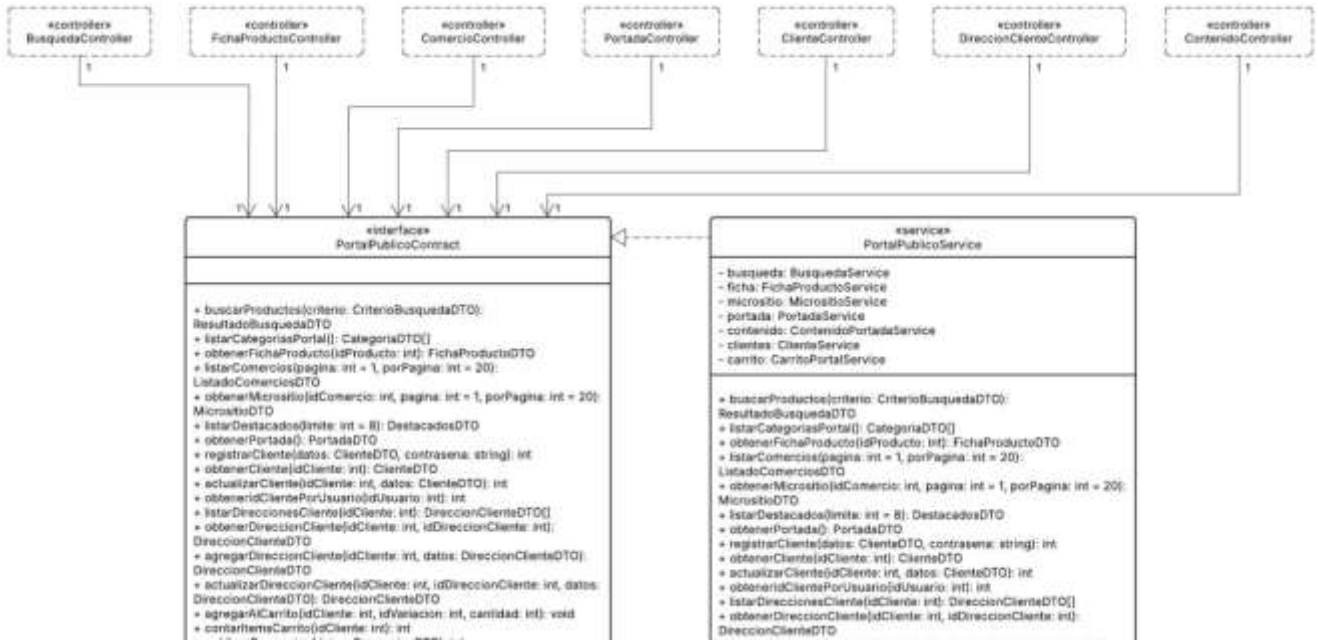


Figura 7.1. Detalle de la implementación del patrón Facade mediante PortalPublicoService y PortalPublicoContract. Fuente: elaboración propia, a partir de la Figura 4.1.

6.2 Adapter

Categoría	Estructural
Problema del módulo que resuelve	Cada módulo externo (M2, M3, M4, M5) publica su propia interfaz PHP de contrato dentro de la misma aplicación (CatalogoServiceInterface de M3, ComerciosServiceInterface de M2, InventarioModuleInterface de M4, CarritoServiceInterface de M5. M1 necesita trabajar con sus propios DTO de dominio (ProductoResumenDTO, FichaProductoDTO, ComercioDTO, DisponibilidadDTO) sin acoplar su lógica de negocio a la firma exacta de cada interfaz externa.
Dónde se aplica	Clases «adapter» `CatalogoClient`, `ComercioClient`, `InventarioClient` y `CarritoClient`. cada una implementando la llamada a su interfaz de contrato correspondiente y traduciendo la respuesta a los DTO de M1.
Justificación	El patrón adapter permite que M1 consuma las funcionalidades de M2, M3, M4, M5 mediante sus contratos publicados, sin depender directamente de sus implementaciones internas. Si alguno de estos módulos publica una nueva versión compatible de su contrato los ajustes necesarios pueden concentrarse en el adaptador correspondiente, sin modificar los servicios de M1.
Alternativa descartada	Que los servicios de aplicación llamen directamente las interfaces de M2/M3/M4/M5 y trabajen con sus tipos de retorno. Se descartó porque acoplaría el diseño interno de M1 a decisiones de otros módulos que SG1 no controla.

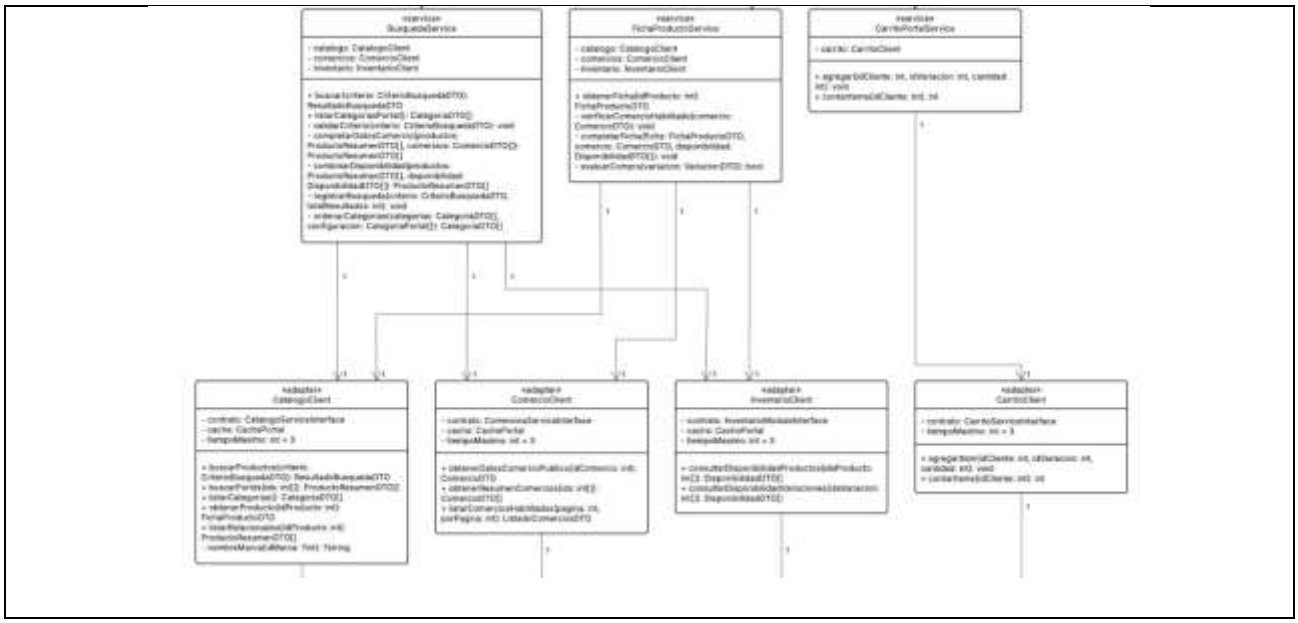


Figura 7.2. Detalle de la implementación del patrón Adapter mediante las clases adaptadoras que conectan los servicios internos de M1 con los contratos de los módulos externos. Fuente: elaboración propia, a partir de la Figura 4.2.

6.3 Observer

Categoría	De comportamiento
Problema del módulo que resuelve	El portal publico necesita reaccionar a los cambios de estado de los comercios y d las existencias que ocurren en M2 Y M4, sin depender exclusivamente de consultas
Dónde se aplica	EventosComercioListener y EventosExistenciaListener, que consumen los eventos publicados por M2 (ComercioAprobado, ComercioSuspendido y ComercioReactivado) y M4 (ExistenciaAgotada y ExistenciaRepuesta).
Justificación	El patrón Observer permite que M1 reaccione a los eventos de negocio mediante sus listeners, actualizando o invalidando la información almacenada en CachePortal según corresponda. Esto reduce la necesidad de consultas adicionales para detectar cambios y mantiene desacoplados los módulos, respetando la comunicación asíncrona establecida en el ADR-005 del Comité.
Alternativa descartada	Consultar continuamente los contratos de M2 y M4 para detectar cambios de estado. Se descartó como mecanismo exclusivo de actualización porque generaría consultas adicionales y mayor dependencia de los módulos externos, cuando los eventos permiten notificar los cambios de manera asíncrona.

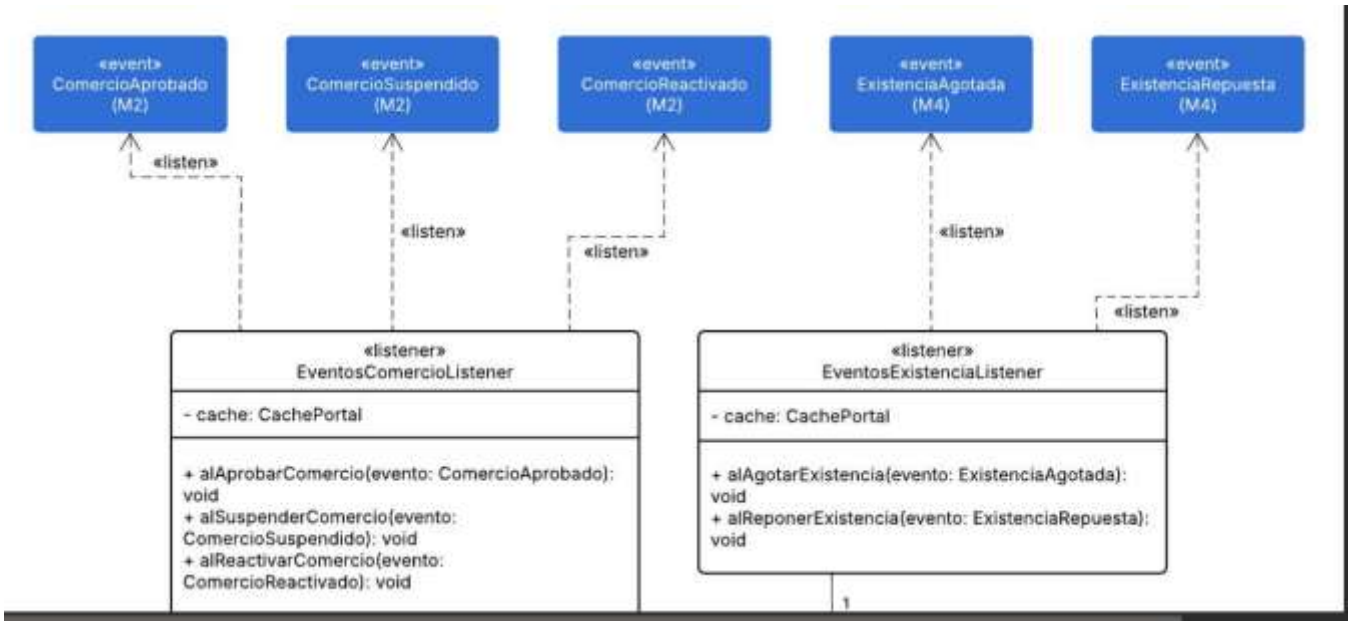


Figura 7.3. Detalle de los eventos consumidos y sus clases Listener, relacionados con la aplicación del patrón Observer. Fuente: elaboración propia, a partir de la Figura 4.8.

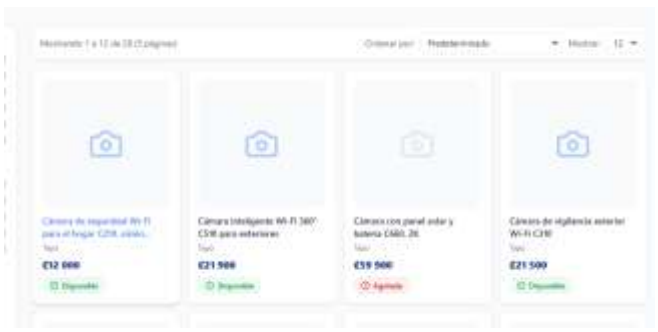
7. Prototipo de interfaz del módulo, validado

7.1 Pantallas del prototipo

Pantalla	Caso de uso	Actor	Ubicación (repositorio o enlace)
Página principal (búsqueda, destacados, promociones, noticias)	CU-002, CU-005, CU-006, CU-007, CU-008	Cliente	Pacifico-online\frontend\src\modules\M1_Portal\views\InicioView.jsx
Resultados de búsqueda con filtros de categoría	CU-002, CU-004	Cliente	resources/js/Pages/M1/ Busqueda.jsx
Ficha detallada de producto	CU-003	Cliente	pacifico-online\frontend\src\modules\M1_Portal\views\FichaProductoView.jsx
Micrositio público de un comercio	CU-009	Cliente	pacifico-online\frontend\src\modules\M1_Portal\views\FichaComercioView.jsx
Formulario de registro de cliente	CU-010	Cliente	pacifico-online\frontend\src\modules\M1_Portal\views\CrearCuentaView.jsx
Estado "agotado"	CU-002, CU-003	Cliente	pacifico-online\frontend\src\modules\M1_Portal\components\TarjetaProducto.jsx

7.2 Validación

Fecha	21/10/2026 → Iván Chavarría
Participantes	Karilyn Cortés Espinoza, Roberth
Método	Prueba de uso mediante recorrido guiado con tareas
Tareas evaluadas	1. Búsqueda de artículos, micrositios 2. Inicio de Sesión 3. Ficha de producto 4. Visualización del micrositio 5. Pantalla de inicio con categorías, micrositios y productos

Hallazgo	Pantalla	Estado
Barra de búsqueda sin funcionalidad		Pendiente
Carrito sin funcionalidad		Pendiente
Categorías sin funcionalidad		pendiente
Es posible ingresar a productos agotados		Pendiente

8. Dobles de prueba de las dependencias externas

Permiten avanzar sin esperar a los otros subgrupos: sustituyen a cada módulo o servicio externo del que depende el módulo.

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M2 Comercios *obtenerDatos ComercioPublico* *obtenerResumen ComerciosLote* *listarComercios Habilitados*	*ComerciosService Interface*, contrato v1.2.1	Stub	**Éxito.** Solo comercios activos; los suspendidos se omiten. **Errores.** ERR-004 comercio inexistente o suspendido, ERR-005 datos inválidos, ERR-006 M2 no responde.	*Dobles/Comercios Stub.php*	M2 publique sus firmas PHP y su servicio.
M3 Catálogo *buscarProductos* *obtenerProducto* *listarCategorias* *listarMarcas* *listar Relacionados*	*CatalogoService Interface*, contrato v1.2.0 (provisional)	Fake	**Éxito.** Búsqueda con filtros y paginación, ficha con variaciones y relacionados. **Errores.** ERR-004 producto inexistente o no visible, ERR-005 filtros inválidos, ERR-006 M3 no responde.	*Dobles/Catalogo Fake.php*	Se ajuste a la v1.3.0 de M3 y su servicio esté disponible.
M4 Inventario *consultar Disponibilidad Lote* *consultar Disponibilidad ProductoLote*	*InventarioService Interface*, contrato v1.1.0	Stub	**Éxito.** Disponible o agotado por variación y por producto. Sin existencia registrada se muestra agotado. **Errores.** ERR-005 lote inválido, ERR-006 M4 no responde.	*Dobles/Inventario Stub.php*	M4 publique *InventarioModule Interface* y su servicio.
M5 Carrito *obtenerCarrito* *agregarItem Carrito*	*PedidosService Interface*, contrato v1.1.0	Fake	**Éxito.** Agrega ítems y los conserva durante la petición. **Errores.** ERR-005 variación inválida o no vendible, ERR-M5-001 sin existencias, ERR-006 M5 no responde.	*Dobles/CarritoFake.php*	El servicio real de M5 esté en el repositorio.

Las rutas son relativas a `app/Services/M1_Portal/`. La falla de cada proveedor (ERR-006) se simula con las variantes de `tests/Doubles/M1/`.

El cambio del doble a la implementación real se hace por configuración, sin modificar el código del módulo. En el archivo `.env` se cambia `M1_M2_MODO`, `M1_M3_MODO`, `M1_M4_MODO` o `M1_M5_MODO` de `doble` a `real`, según `config/integraciones.php`.